STAGE 1

Title of the Project:

Analysis and Impact Assessment of Common Software Security Vulnerabilities Using CWE Standards

Overview:

In today's digital world, software applications are widely used in industries such as banking, healthcare, education, government systems, and e-commerce. As the dependency on software systems increases, security vulnerabilities also become a major concern. Cyber attackers continuously search for weaknesses in applications to exploit them for unauthorized access, data theft, or service disruption. Therefore, identifying and understanding common software vulnerabilities is essential for developers, security professionals, and organizations.

This project focuses on analyzing common software vulnerabilities based on the Common Weakness Enumeration (CWE) standard. CWE is a well-known classification system developed to identify and categorize weaknesses in software and hardware. These weaknesses can potentially lead to serious security issues if they are not properly addressed during software development or deployment. By studying these vulnerabilities, developers can understand the root causes of security problems and implement proper mitigation techniques.

The primary objective of this project is to study ten common vulnerabilities that frequently appear in modern applications. Each vulnerability will be analyzed in terms of its CWE identification number, description, security category according to OWASP or SANS, and its potential business impact. Understanding the business impact is important because vulnerabilities do not only affect technical

systems but also influence financial stability, organizational reputation, customer trust, and legal compliance.

This project will examine vulnerabilities such as Null Pointer Dereference, Sensitive Information Exposure in Logs, Reliance on Cookies without Validation, Buffer Overflow, SQL Injection, Cross-Site Scripting, Improper Input Validation, Broken Authentication, Insecure Direct Object References, and Missing Authorization Checks. These vulnerabilities are commonly found in web applications, mobile applications, and enterprise software systems.

Another important goal of this project is to demonstrate how poor coding practices, lack of input validation, improper error handling, and insecure authentication mechanisms can create serious security risks. By documenting these weaknesses, developers and security teams can better understand how attackers exploit these vulnerabilities and how proper secure coding practices can prevent them.

This project also highlights the relationship between CWE vulnerabilities and OWASP/SANS security categories, which are widely used frameworks in cybersecurity. OWASP provides the well-known OWASP Top 10 list, which describes the most critical security risks in web applications. By mapping vulnerabilities to these categories, the project provides a clearer understanding of how real-world attacks occur.

In conclusion, this project provides a comprehensive overview of common software vulnerabilities and their impact on organizations. By analyzing different CWE vulnerabilities and their business implications, this project aims to raise awareness about secure software development practices and the importance of implementing strong security controls in modern applications.

List of teammates:

S.no	name	collage	contact
1	Deepika V	AVP College of Arts and Science	deepikavenkatesan121@gmail.com
2	Charumathi A	AVP College of Arts and Science	Charumathiarumugam12@gmail.com
3	Priya dharshini B	AVP College of Arts and Science	Priyadharshinibakiyaraj906@gmail.com
4	Moulisha S	AVP College of Arts and Science	moulishaselvakumar@gmail.com
5	Harshini N	AVP College of Arts and Science	murukesvarinakaraj@gmail.com

List of Vulnerability Table

S.no	Vulnerability Name	CWE/CVE No
1	NULL Pointer Dereference	CWE-476
2	Insertion of Sensitive Information into Log File	CWE-532
3	Reliance on Cookies without Validation	CWE-784
4	Buffer Overflow	CWE-120
5	SQL Injection	CWE-89
6	Cross-Site Scripting (XSS)	CWE-79
7	Improper Input Validation	CWE-20
8	Broken Authentication	CWE-287
9	Insecure Direct Object Reference	CWE-639
10	Missing Authorization Check	CWE-862

REPORT

1.Vulnerability Name

Sensitive Information Disclosure in Source Code Comments

CWE / CVE

The identified vulnerability is classified as CWE-615, which is maintained by the MITRE Corporation.

This classification describes a security weakness where developers leave confidential information inside comments within the source code. These comments may expose internal details about the application's design or implementation that should not be accessible to users or attackers.

OWASP / SANS Category

According to the OWASP Foundation, this issue is commonly associated with Sensitive Data Exposure and sometimes related to Security Misconfiguration in the OWASP Top 10 security risks.

These categories highlight the importance of protecting confidential information within applications and ensuring that internal implementation details are not unnecessarily exposed

Description

Sensitive Information in Source Code Comments is a security weakness that arises when developers leave confidential or internal information within comments embedded in the application's codebase. Comments are commonly used to document code behavior, provide instructions to developers, or explain complex algorithms. While this practice improves readability and collaboration during development, it may also unintentionally reveal information that attackers can exploit.

For example, developers may leave comments containing internal file structures, test URLs, API endpoints, database schema details, or debugging instructions that were intended only for development purposes. If the application exposes its client-side source code or scripts to users, these comments can be easily viewed through browser developer tools or by inspecting downloaded files.

Attackers frequently examine source code comments because they may contain clues about hidden functionality or unfinished features within the system. By analyzing these comments, attackers can map out the internal architecture of the application, identify administrative interfaces, and determine potential entry points for further attacks.

In some cases, comments may reveal outdated code fragments or references to deprecated features that still exist in the system. These fragments may point attackers toward vulnerable components that were not properly removed from the application. As a result, even though comments themselves do not execute code, they can indirectly contribute to security breaches by exposing information that assists attackers during the reconnaissance phase of an attack.

Business Impact

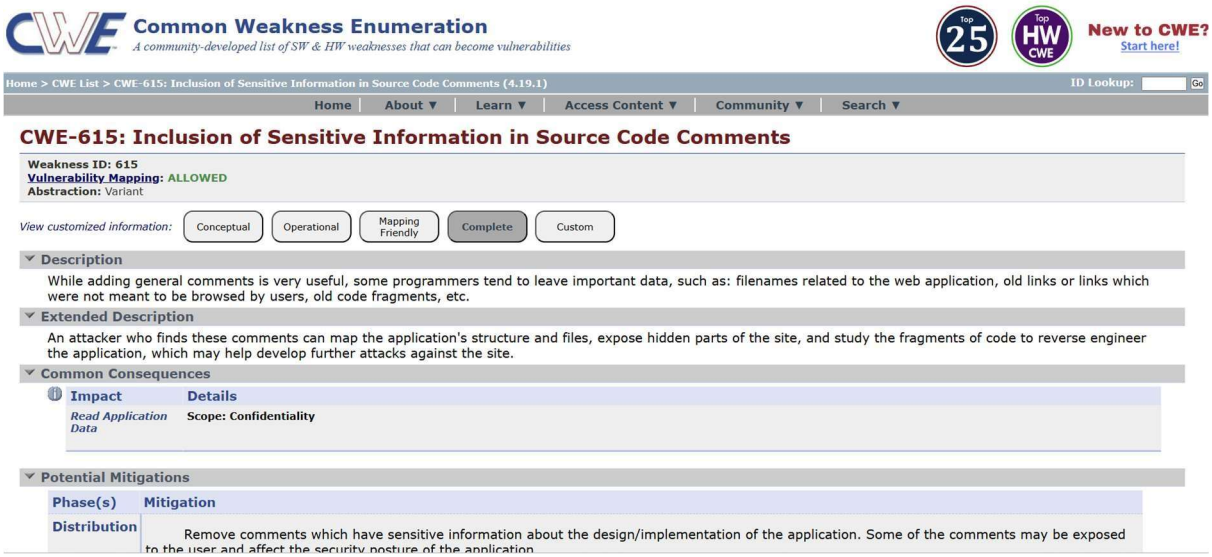
The presence of sensitive information within source code comments can significantly increase the risk of security breaches for an organization. Although comments are intended only for developers, their exposure in publicly accessible code may reveal important technical details about the internal structure of the application. Attackers can use this information to better understand how the system operates, which components interact with each other, and where potential weaknesses might exist. By analyzing these internal details, attackers can perform more targeted reconnaissance and reduce the time required to discover exploitable vulnerabilities within the system.

Another major concern is that comments may disclose hidden resources or internal endpoints that were never meant to be accessed by external users. For instance, comments might reference administrative login pages, testing environments, development server addresses, or temporary APIs used during the development process. If attackers discover these hidden paths, they may attempt to access them directly, potentially bypassing security controls or authentication mechanisms. This can lead to unauthorized access to sensitive data, modification of application functionality, or disruption of normal system operations.

In addition to technical risks, the exposure of sensitive information through comments can have serious business consequences. If attackers successfully exploit the information revealed in comments, the organization may suffer data breaches involving customer information, financial records, or proprietary business data. Such incidents can result in financial losses, legal liabilities, regulatory penalties, and reputational damage. Customers and stakeholders may lose trust in the organization's ability to protect their data, which can affect long-term business relationships and brand reputation.

Furthermore, organizations that fail to follow secure coding practices may face compliance issues with security standards and regulations. Many cybersecurity frameworks require organizations to implement strict controls over sensitive information and ensure that internal system details are not publicly exposed. Leaving sensitive data in source code comments may violate these requirements and lead to audit failures or compliance penalties.

For these reasons, it is essential for development teams to implement secure coding practices that prevent the inclusion of sensitive information in source code comments. Regular code reviews, automated security scans, and proper development guidelines can help ensure that confidential details are removed before applications are deployed to production environments. By addressing this issue proactively, organizations can reduce the likelihood of information leakage and strengthen the overall security of their software systems.



REPORT

2.Vulnerability Name

Missing Authentication for Critical Function

CWE / CVE

The vulnerability is categorized as CWE-306, which is maintained by the MITRE Corporation. This weakness occurs when an application fails to verify the identity of users before allowing them to access sensitive or critical system operations.

OWASP / SANS Category

According to the OWASP Foundation, this vulnerability is closely related to Identification and Authentication Failures, one of the major risks listed in the OWASP Top 10 security risks for modern web applications.

Description

The vulnerability known as Missing Authentication for Critical Function occurs when an application allows users to access sensitive or important system operations without verifying their identity. Authentication mechanisms are designed to ensure that only authorized users can perform certain actions within a system. When these mechanisms are missing or improperly implemented, attackers may be able to access restricted features without providing valid credentials.

In many web applications, certain functions are considered critical because they allow users to perform actions that can significantly impact the system. Examples include modifying user accounts, accessing confidential data, updating application settings, managing financial transactions, or controlling administrative features. These functions must always be protected by authentication and authorization checks to ensure that only legitimate users can perform them.

However, developers sometimes overlook authentication requirements for certain endpoints or internal APIs. This may occur when developers assume that the function will only be accessed by trusted users or internal systems. In reality, attackers can discover these unprotected endpoints through various techniques such as inspecting web traffic, reviewing client-side scripts, or performing automated vulnerability scans.

Once attackers identify an endpoint that lacks authentication, they may exploit it to perform unauthorized actions. For example, an attacker could access administrative functionality without logging in, modify application configurations, retrieve sensitive information, or

manipulate data stored within the system. This vulnerability can therefore serve as a gateway for more severe attacks, especially when combined with other weaknesses in the application.

Furthermore, missing authentication controls can allow automated bots or malicious users to repeatedly access system resources, potentially leading to system misuse, data corruption, or service disruption. For these reasons, this vulnerability is considered a serious security risk and must be addressed during the development and deployment phases of the application lifecycle.

Business Impact

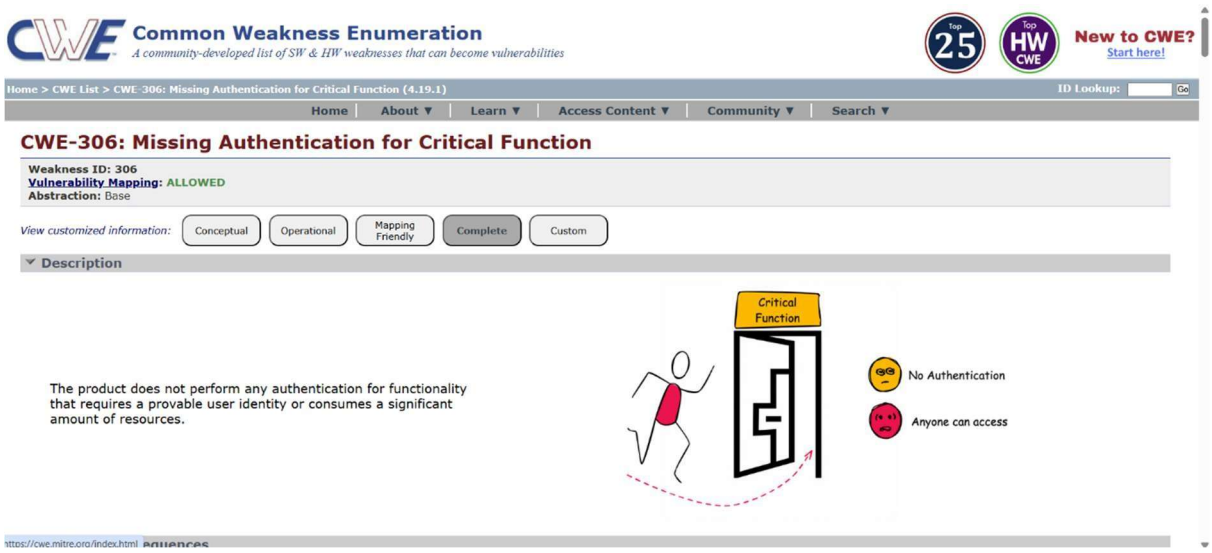
The absence of proper authentication mechanisms for critical system functions can have severe consequences for an organization's security and operational stability. When sensitive operations are accessible without verifying user identity, attackers may gain unauthorized control over important components of the system. This can lead to unauthorized data access, modification of system configurations, and disruption of normal business operations. In environments where applications handle confidential information such as customer data, financial records, or proprietary business information, the consequences of such unauthorized access can be particularly damaging.

Another major concern is that attackers can exploit these unprotected functions to escalate their privileges within the system. For example, if administrative features are accessible without authentication, an attacker may be able to create new user accounts, modify existing permissions, or disable security controls. Such actions can compromise the integrity of the entire application environment and allow attackers to maintain persistent access to the system.

From a financial perspective, organizations may suffer significant losses if attackers exploit authentication vulnerabilities. Unauthorized access to financial systems can lead to fraudulent transactions, theft of sensitive data, or manipulation of critical records. In addition, organizations may incur costs related to incident response, system recovery, and implementation of additional security controls after a breach has occurred.

Beyond financial damage, organizations may also experience reputational harm and loss of customer trust. Customers expect companies to protect their personal information and maintain secure systems. If a security breach occurs due to weak authentication practices, it may negatively affect the organization's public image and lead to decreased customer confidence. In some industries, regulatory bodies also require organizations to implement strict authentication controls to protect sensitive data, and failure to comply with these regulations may result in legal penalties or compliance violations.

Therefore, ensuring that all critical system functions are protected by strong authentication mechanisms is essential for maintaining the security, reliability, and trustworthiness of modern web applications. Organizations must adopt secure development practices, conduct regular security assessments, and implement robust authentication frameworks to prevent unauthorized access to sensitive operations.



REPORT

3.Vulnerability Name

Reliance on Cookies without Validation and Integrity Checking

CWE / CVE

The vulnerability is categorized as CWE-784, which is maintained by the MITRE Corporation. This weakness occurs when applications rely on cookie values to make security decisions without verifying the integrity or authenticity of the data contained within the cookie.

OWASP / SANS Category

According to the OWASP Foundation, this vulnerability is closely related to Identification and Authentication Failures and improper session management practices in web applications.

Description

The vulnerability known as Reliance on Cookies without Validation and Integrity Checking occurs when an application trusts data stored in browser cookies without verifying whether that data has been modified by the user. Cookies are stored on the client side, meaning they reside in the user's browser rather than on the server. Because of this, attackers can easily access and modify cookie values using browser developer tools or specialized tools designed for manipulating web traffic.

In many web applications, cookies are used to store important information such as session identifiers, user roles, authentication status, or preferences. If the application uses these cookie values to determine whether a user is authenticated or authorized to access certain features, attackers may attempt to manipulate them to gain unauthorized privileges.

For example, an application might store a value in a cookie indicating that the user is an administrator. If this cookie value is not properly validated or protected using cryptographic techniques, an attacker could manually modify the cookie to change their role from a normal user to an administrator. Once this modification is accepted by the server, the attacker may gain access to restricted functionality such as administrative dashboards, configuration settings, or sensitive data.

Attackers can also intercept cookies during communication if proper transport security mechanisms such as HTTPS are not implemented. In such cases, attackers could steal valid

session cookies and impersonate legitimate users. This type of attack, commonly known as session hijacking, can allow attackers to bypass authentication mechanisms entirely.

Furthermore, relying on unvalidated cookies for security decisions undermines the trust model of the application. Since cookies originate from the client side, they should never be trusted without verification. Secure applications typically store sensitive authentication and authorization data on the server and use cookies only as references to server-side session information. Without such safeguards, attackers can manipulate client-side data to bypass security restrictions and gain unauthorized access.

Business Impact

The misuse of cookies for making security decisions without proper validation can expose organizations to significant cybersecurity risks. When applications rely on client-controlled data such as cookies to determine user privileges or authentication status, attackers may manipulate these values to gain unauthorized access to sensitive resources. This can allow attackers to bypass authentication controls, impersonate legitimate users, or escalate their privileges within the system.

One of the most serious consequences of this vulnerability is unauthorized access to restricted areas of the application. If attackers can modify cookies to elevate their privileges, they may gain access to administrative interfaces or sensitive system functions. These interfaces often provide the ability to manage user accounts, modify system configurations, or access confidential data. Unauthorized access to such features can compromise the integrity and confidentiality of the entire system.

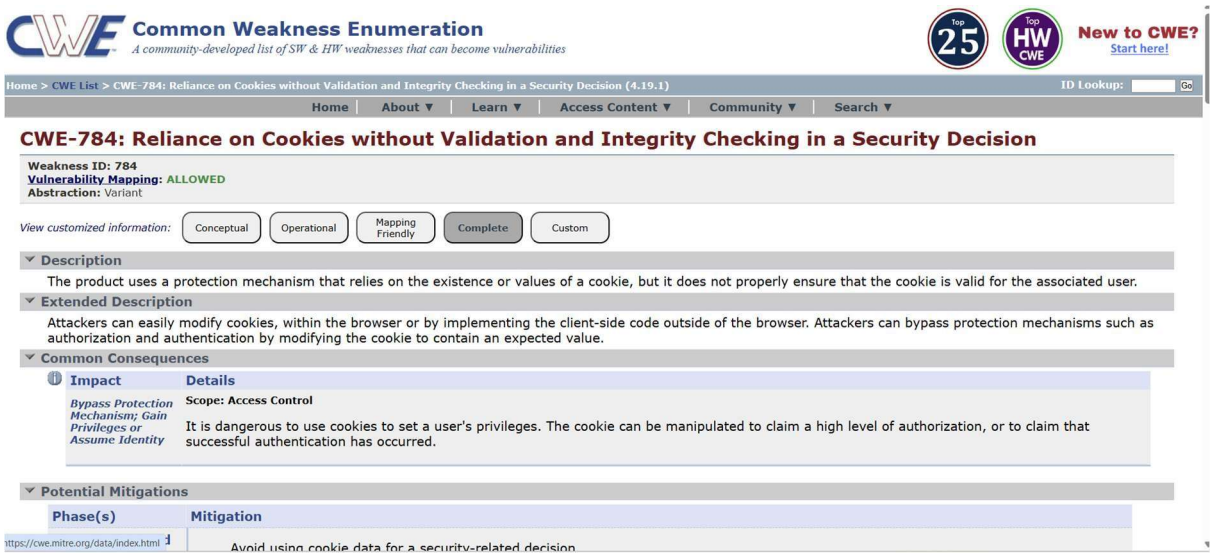
Another major risk associated with this vulnerability is session hijacking. If cookies are not properly secured or validated, attackers may intercept or steal session cookies and use them to impersonate legitimate users. Once attackers gain access to a valid session, they may perform actions on behalf of the victim without needing to know their login credentials. This can lead to unauthorized transactions, data theft, or manipulation of user accounts.

From a business perspective, the consequences of cookie manipulation attacks can be severe. Organizations may suffer financial losses due to fraud, data breaches, or operational disruptions. In addition, compromised systems may expose sensitive customer information

such as personal data, financial records, or login credentials. This can lead to legal liabilities, regulatory penalties, and significant damage to the organization's reputation.

Loss of customer trust is another critical concern. Users expect organizations to protect their personal data and maintain secure systems. If a vulnerability allows attackers to bypass authentication controls or manipulate session data, customers may lose confidence in the organization's ability to safeguard their information. This can result in reduced customer engagement, negative publicity, and long-term reputational harm.

For these reasons, organizations must implement secure session management practices and ensure that cookie data cannot be trusted without proper validation. By enforcing strong security controls and validating all client-side data, organizations can reduce the risk of unauthorized access and protect both their systems and their users from potential cyber threats.



REPORT

4.Vulnerability Name

Sensitive Information Exposure in Log Files

CWE / CVE

The identified vulnerability is categorized as CWE-532, which is maintained by the MITRE Corporation.

This weakness describes a situation where sensitive or confidential data is written into application log files, making it possible for unauthorized individuals to access and misuse the information contained in those logs.

OWASP / SANS Category

According to the OWASP Foundation, this vulnerability is related to Sensitive Data Exposure and improper security logging practices in web applications.

Description

The vulnerability known as Insertion of Sensitive Information into Log File occurs when an application records confidential data in log files during normal operation. Logging systems are designed to capture information about application events such as user login attempts, error messages, system activity, and performance monitoring. However, if sensitive data is logged without proper consideration, it may unintentionally expose confidential information to unauthorized parties.

Sensitive information that may appear in log files includes user passwords, authentication tokens, credit card numbers, session identifiers, personal user data, or system configuration details. This data may be logged during authentication processes, error handling routines, debugging operations, or transaction monitoring activities. If the application records this information in plain text within log files, it becomes vulnerable to unauthorized access.

Attackers who gain access to log files can analyze them to extract sensitive information about users or system operations. For example, if passwords or session tokens are recorded in logs, attackers may use them to impersonate legitimate users or gain unauthorized access to the system. Similarly, log files may reveal internal system structures, database queries, or API endpoints that could assist attackers in identifying additional vulnerabilities.

Another risk arises when logs are shared across different systems or transmitted to centralized monitoring platforms. If log data is transmitted without encryption or stored in insecure

environments, sensitive information may be intercepted or accessed by unauthorized individuals. Additionally, logs often remain stored for extended periods of time, increasing the likelihood that sensitive information may be exposed if the system is compromised.

Therefore, logging practices must be carefully designed to ensure that only necessary information is recorded while protecting confidential data from exposure. Sensitive fields should be masked, encrypted, or completely excluded from log entries to prevent attackers from exploiting log data.

Business Impact

The presence of sensitive information within log files can create serious security risks for organizations. Log files often contain detailed records of system activity, and if these logs include confidential information such as passwords, session tokens, or personal user data, attackers may be able to exploit them to gain unauthorized access to systems or compromise user accounts. Because logs are frequently stored for long periods of time and may be accessible to multiple systems or administrators, they represent a valuable target for attackers seeking to extract sensitive information.

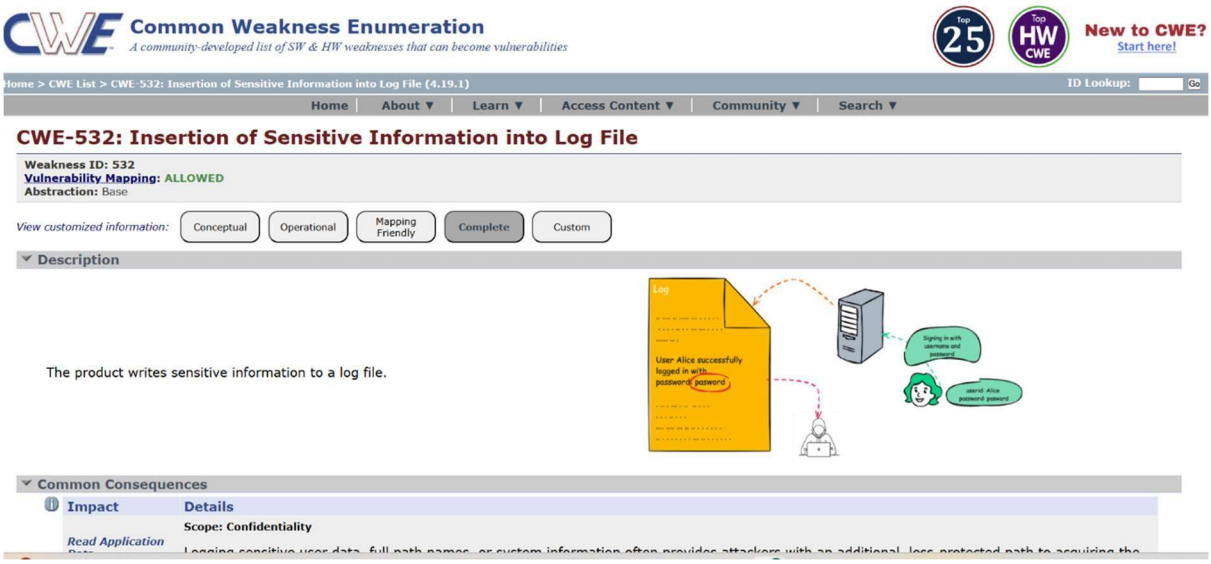
One major consequence of this vulnerability is the potential exposure of user credentials or authentication tokens. If login attempts or authentication processes are logged without proper sanitization, attackers who gain access to these logs may retrieve valid credentials and use them to impersonate legitimate users. This could allow attackers to bypass authentication mechanisms and access sensitive areas of the application without detection. Such unauthorized access may result in data theft, manipulation of records, or disruption of business operations.

Another significant risk involves the leakage of personal or financial information. Applications that process sensitive customer data may inadvertently record details such as personal identification information, financial transactions, or payment data within log files. If these logs are exposed, attackers could collect large amounts of sensitive user information, potentially leading to identity theft, financial fraud, or other malicious activities. In addition to harming affected users, such incidents can severely damage the reputation of the organization responsible for the data.

Organizations may also face regulatory and legal consequences if sensitive data is improperly stored in log files. Many data protection regulations require companies to implement strict safeguards for handling personal and financial information. Failure to protect this data can result in compliance violations, legal penalties, and financial liabilities. Regulatory investigations following a data breach may also require organizations to invest significant resources in remediation and security improvements.

Beyond financial and legal consequences, the exposure of sensitive information through log files can damage customer trust and public reputation. Customers expect organizations to protect their data and implement strong security practices. If sensitive information is leaked due to poor logging practices, customers may lose confidence in the organization's ability to safeguard their personal information. This loss of trust can have long-term effects on customer relationships, brand reputation, and business growth.

For these reasons, organizations must implement secure logging practices and ensure that sensitive information is never recorded in logs unless absolutely necessary. By carefully controlling what data is logged and implementing proper access controls, encryption, and monitoring mechanisms, organizations can reduce the risk of information leakage and protect both their systems and their users from potential cyber threats.



REPORT

5.Vulnerability Name

NULL Pointer Dereference

CWE / CVE

This vulnerability is classified as CWE-476 in the Common Weakness Enumeration system maintained by the MITRE Corporation.

CWE-476 refers to situations where a software application attempts to dereference a pointer that has not been properly initialized or has been assigned a NULL value. Because NULL represents an invalid or empty memory reference, attempting to access it results in program failure or system crashes. OASP / SANS Category

According to secure development guidance from the OWASP Foundation, NULL pointer dereference issues fall under software implementation vulnerabilities that arise from improper input validation, insufficient error handling, or unsafe memory management practices.

These vulnerabilities are typically associated with secure coding weaknesses that can affect application availability and reliability.

Description

The NULL Pointer Dereference vulnerability occurs when a program attempts to access memory through a pointer that is set to NULL instead of referencing a valid memory address. In programming, a pointer is used to store the memory location of a variable or object. If the pointer does not point to a valid location but the program attempts to use it, the system cannot retrieve or modify the data as expected.

This situation often arises when memory allocation fails or when a pointer variable is declared but not properly initialized. Developers may also inadvertently assign NULL values to pointers

during program execution without implementing validation checks before accessing the pointer. When the program later attempts to dereference the pointer, the operating system detects that the pointer refers to an invalid memory location and terminates the process.

In many software environments, dereferencing a NULL pointer causes an application crash. This crash may interrupt normal system operations and affect users who depend on the software for critical tasks. In server environments or web applications, repeated crashes caused by this vulnerability can disrupt services and reduce system availability.

Although NULL pointer dereference vulnerabilities are often considered stability issues rather than direct security flaws, attackers can exploit them to trigger denial-of-service attacks. By carefully crafting input data or triggering specific execution paths within the application, attackers may intentionally cause the program to dereference a NULL pointer, resulting in repeated system failures.

Additionally, in some rare cases involving kernel-level or embedded systems, NULL pointer dereference vulnerabilities may be exploited to gain elevated privileges or execute arbitrary code if memory protections are not properly enforced. For this reason, software developers must treat pointer-related vulnerabilities with serious attention and implement strong validation mechanisms within the application.

Business Impact

The presence of NULL pointer dereference vulnerabilities can significantly impact the reliability and stability of software systems. When applications crash due to invalid memory access, users may experience unexpected interruptions while performing critical operations. For organizations that rely on software services for daily business activities, such interruptions can reduce productivity, disrupt workflows, and lead to operational inefficiencies.

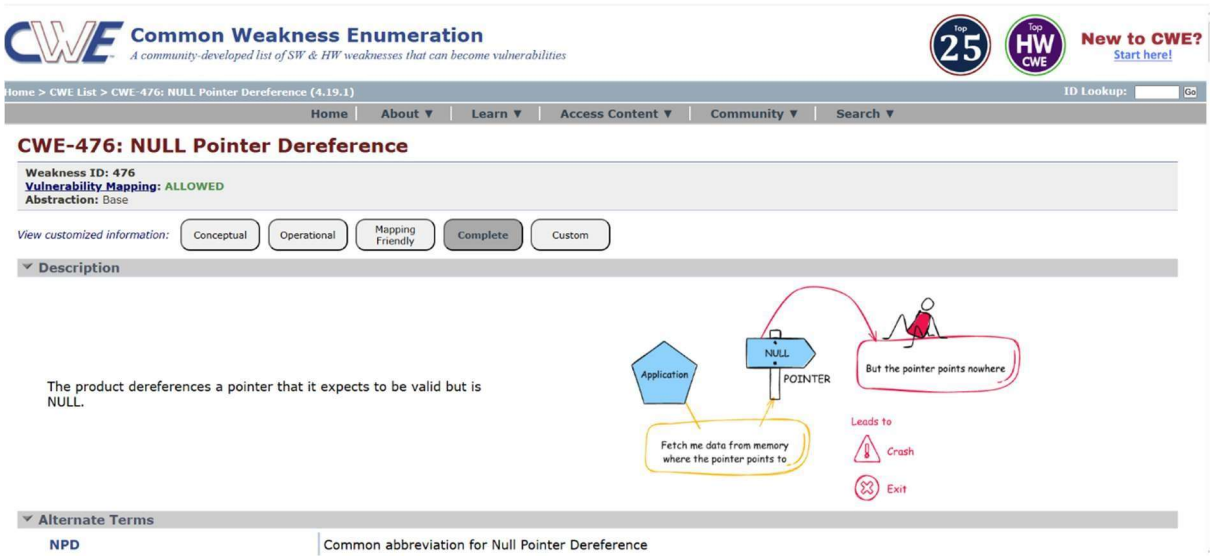
One major consequence of this vulnerability is system downtime. If a server application crashes due to a NULL pointer dereference error, the service may become unavailable until it is restarted or repaired. In high-availability environments such as financial platforms, ecommerce websites, or enterprise applications, even short periods of downtime can result in financial losses and damage to the organization’s reputation.

Another potential business risk involves denial-of-service attacks. Attackers who discover a NULL pointer dereference vulnerability may intentionally trigger it repeatedly in order to crash the application. By continuously exploiting the vulnerability, attackers can prevent legitimate users from accessing the system. This type of attack can severely impact customer experience and may require significant time and resources to investigate and mitigate.

Organizations may also face increased maintenance and development costs when dealing with such vulnerabilities. Debugging memory-related errors can be complex and timeconsuming, especially in large software systems with extensive codebases. Developers may need to analyze system logs, trace memory operations, and redesign portions of the code to prevent the vulnerability from occurring again.

Furthermore, repeated application crashes may negatively affect customer trust and brand reputation. Users expect modern applications to function reliably without unexpected failures. If customers frequently experience system crashes or service interruptions, they may lose confidence in the organization's ability to provide stable and secure services.

For these reasons, organizations must adopt secure development practices and implement thorough testing procedures to identify pointer-related vulnerabilities before deploying software into production environments. Proper input validation, error handling, memory management techniques, and code reviews are essential to preventing NULL pointer dereference issues and ensuring the long-term reliability of software systems.



.

REPORT

6.Vulnerability Name

Persistent Cookies Containing Sensitive Information

CWE/CVE

The vulnerability identified in this assessment corresponds to CWE-539: Use of

Persistent Cookies Containing Sensitive Information, which is a security weakness recognized within the Common Weakness Enumeration (CWE) framework. CWE-539 describes a scenario where sensitive information is stored inside persistent cookies that remain on the client device even after the browser session has ended. Persistent cookies differ from session cookies in that they are not automatically deleted when the browser closes and can remain stored for a long period of time depending on their expiration settings. Because these cookies are stored locally on the user's system, they may be accessed by attackers who gain access to the device, compromise the browser environment, or exploit other vulnerabilities that allow them to read client-side data. If sensitive information such as authentication credentials, session tokens, or personal user data is stored within these cookies without proper protection mechanisms such as encryption or secure attributes, attackers may retrieve this information and use it to impersonate legitimate users or gain unauthorized access to restricted areas of the application. This vulnerability highlights the importance of secure data storage practices and proper session management mechanisms in web application development.

OWASP/SANS Category

This vulnerability is closely related to security risks described in the OWASP Top 10

2021, particularly under A02: Cryptographic Failures and A07: Identification and Authentication Failures. Cryptographic failures occur when sensitive data is not properly protected during storage or transmission, while identification and authentication failures involve weaknesses in session management and authentication mechanisms that may allow

attackers to compromise user identities. Storing sensitive information in persistent cookies without proper protection mechanisms can expose critical authentication data and increase the likelihood of unauthorized access.

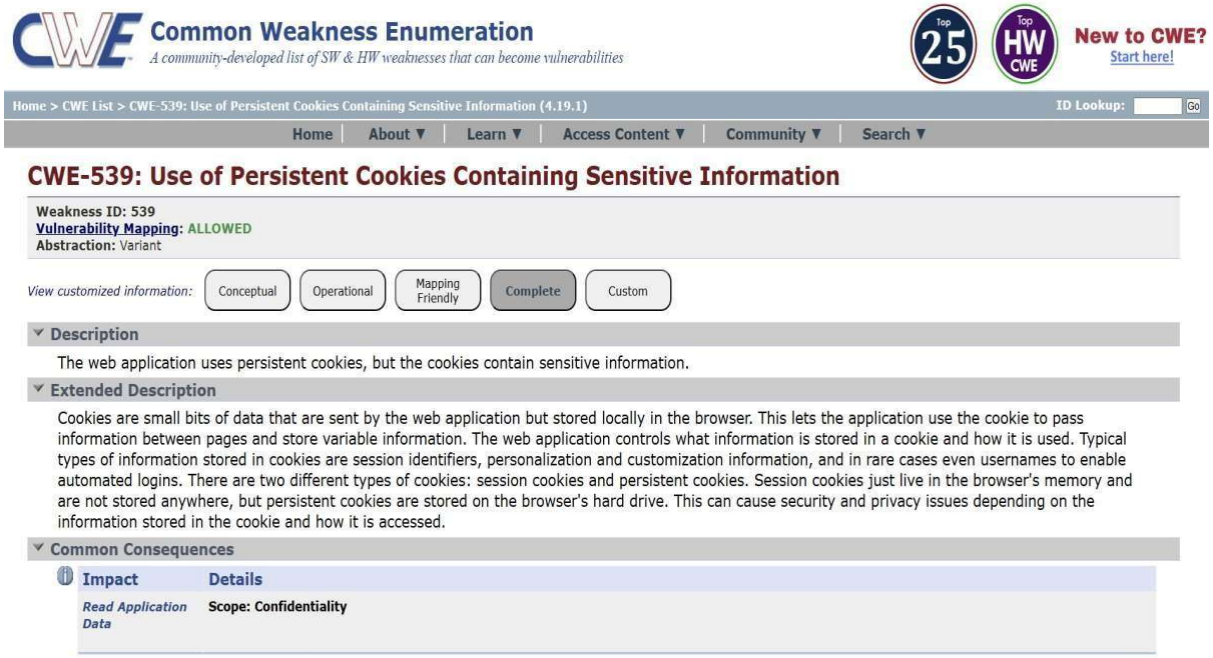
Additionally, this vulnerability aligns with issues highlighted in the SANS Top 25 Most Dangerous Software Errors, which identifies improper handling of sensitive information and insecure storage of data as some of the most common and dangerous programming errors in modern software applications. Organizations that fail to properly secure cookies and other client-side storage mechanisms may unintentionally expose confidential data to attackers, increasing the risk of system compromise and data breaches.

Description

Cookies are small pieces of information that are created by web servers and stored within a user's web browser to maintain stateful interactions between the client and the server. They play an essential role in modern web applications by enabling features such as user authentication, session management, personalization, and tracking of user preferences. There are generally two types of cookies used in web applications: session cookies and persistent cookies. Session cookies are temporary and exist only during the active browser session, meaning they are automatically deleted when the browser is closed. Persistent cookies, on the other hand, are stored on the user's device for a predefined period of time and remain accessible even after the browser is closed. While persistent cookies can improve usability by remembering user preferences and login states, storing sensitive information in them can introduce serious security risks. If sensitive data such as authentication tokens, user identifiers, personal information, or session IDs is stored within persistent cookies, attackers may retrieve this information through various attack techniques including malware infections, cross-site scripting attacks, browser vulnerabilities, or direct access to the user's device. Once the sensitive data is obtained, attackers may use it to impersonate legitimate users, bypass authentication mechanisms, or gain unauthorized access to protected resources within the application. This vulnerability demonstrates the importance of ensuring that sensitive data is never stored in persistent cookies unless it is properly protected and secured using appropriate security mechanisms.

Business Impact

The presence of persistent cookies containing sensitive information can have significant consequences for organizations, users, and the overall security posture of the application. If attackers successfully exploit this vulnerability, they may gain unauthorized access to user accounts and sensitive system resources, leading to potential data breaches and exposure of confidential information. Such incidents can result in financial losses, regulatory penalties, and legal consequences for organizations that fail to adequately protect user data. Additionally, compromised user accounts may be used by attackers to perform fraudulent activities, manipulate application data, or carry out further attacks within the system. Beyond the direct technical impact, security breaches caused by this vulnerability can significantly damage an organization's reputation and erode customer trust. Users expect their personal and sensitive information to be handled securely, and any failure to protect this data may result in loss of business, negative publicity, and long-term reputational harm. Furthermore, organizations may face compliance violations with data protection regulations such as GDPR, HIPAA, or other industry-specific standards if sensitive information is improperly stored and exposed. Therefore, addressing this vulnerability is critical to maintaining strong security practices, protecting user privacy, and ensuring the long-term reliability and credibility of the web application.



Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-539: Use of Persistent Cookies Containing Sensitive Information (4.19.1)

ID Lookup: Go

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-539: Use of Persistent Cookies Containing Sensitive Information

Weakness ID: 539
Vulnerability Mapping: ALLOWED
Abstraction: Variant

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The web application uses persistent cookies, but the cookies contain sensitive information.

Extended Description

Cookies are small bits of data that are sent by the web application but stored locally in the browser. This lets the application use the cookie to pass information between pages and store variable information. The web application controls what information is stored in a cookie and how it is used. Typical types of information stored in cookies are session identifiers, personalization and customization information, and in rare cases even usernames to enable automated logins. There are two different types of cookies: session cookies and persistent cookies. Session cookies just live in the browser's memory and are not stored anywhere, but persistent cookies are stored on the browser's hard drive. This can cause security and privacy issues depending on the information stored in the cookie and how it is accessed.

Common Consequences

Impact	Details
Read Application Data	**Scope: Confidentiality**

REPORT

7. Vulnerability Name

Unexpected Status Code or Return Value

CWE / CVE CWE-394

OWASP / SANS Category Mapped conceptually to OWASP Foundation Top 10 – A03: Injection / Improper Error Handling

Description

The Unexpected Status Code or Return Value vulnerability occurs when an application does not properly validate or handle return values from functions, APIs, system calls, or external services. In software systems, many operations return specific status codes or values that inform the program whether the operation was successful, partially successful, or encountered an error condition. If the application only checks for expected values and ignores other possible valid return values, it may fail to properly handle unusual but legitimate responses.

This situation may cause the application to behave incorrectly or enter an unexpected state. For example, a function may return a value indicating that an operation completed with

warnings, partial success, or a special condition. If the application assumes only success or failure outcomes and does not account for other responses, it may incorrectly process data or continue execution in an unsafe manner. In some cases, attackers may exploit this weakness by triggering specific conditions that cause the system to receive unusual return values, potentially bypassing security checks or causing the application to behave unpredictably.

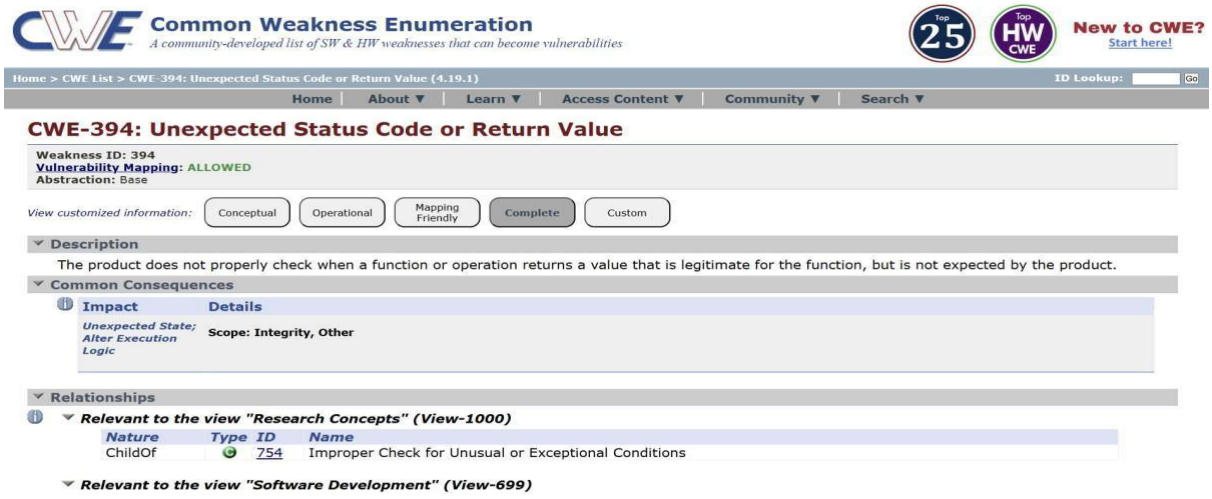
This vulnerability commonly occurs in complex systems that interact with external APIs, databases, network services, or operating system functions. Inadequate error handling, lack of input validation, and incomplete understanding of function behaviors are common causes. Proper defensive programming techniques, comprehensive error checking, and validation of all possible return values are necessary to prevent such vulnerabilities.

Business Impact

The business impact of the Unexpected Status Code or Return Value vulnerability can be significant because it affects the reliability, integrity, and security of software systems. When applications fail to properly handle unexpected responses from functions or external systems, they may behave unpredictably or perform incorrect operations. This can lead to system crashes, corrupted data, or incorrect transaction processing, which directly affects the quality and reliability of the organization's services.

In security-sensitive environments such as financial systems, healthcare platforms, or enterprise applications, improper handling of return values could allow attackers to manipulate system behavior or bypass important validation mechanisms. This may result in unauthorized access to sensitive information, incorrect authorization decisions, or disruption of critical services. Additionally, unpredictable system behavior may lead to operational downtime, affecting business productivity and customer trust.

Organizations that rely on complex software infrastructures may also face increased costs associated with troubleshooting, incident response, and system recovery when unexpected errors occur. If such issues lead to service outages or data integrity problems, the organization may experience reputational damage and potential financial losses. Therefore, ensuring robust error handling and comprehensive validation of all return values is essential for maintaining system stability, protecting sensitive data, and supporting reliable business operations.



REPORT

8. Vulnerability Name

Insufficient Logging

CWE / CVE CWE-778

OWASP / SANS Category Mapped to OWASP Foundation Top 10 – A09: Security Logging and Monitoring Failures

Description

The Insufficient Logging vulnerability occurs when an application or system fails to properly record important security-related events or logs them without sufficient details for analysis. Logging is a critical security mechanism that helps track system activities and detect malicious behavior. However, when logs are incomplete, missing, or not generated for key events, it becomes extremely difficult to monitor the system effectively or respond to security incidents.

For example, if an application does not log failed login attempts, privilege changes, or suspicious user activities, administrators will not be able to detect brute-force attacks, unauthorized access attempts, or abnormal system behavior. Even when logs exist, they may lack important contextual information such as timestamps, user identifiers, IP addresses, or

event descriptions, making it difficult to reconstruct the sequence of events during an investigation.

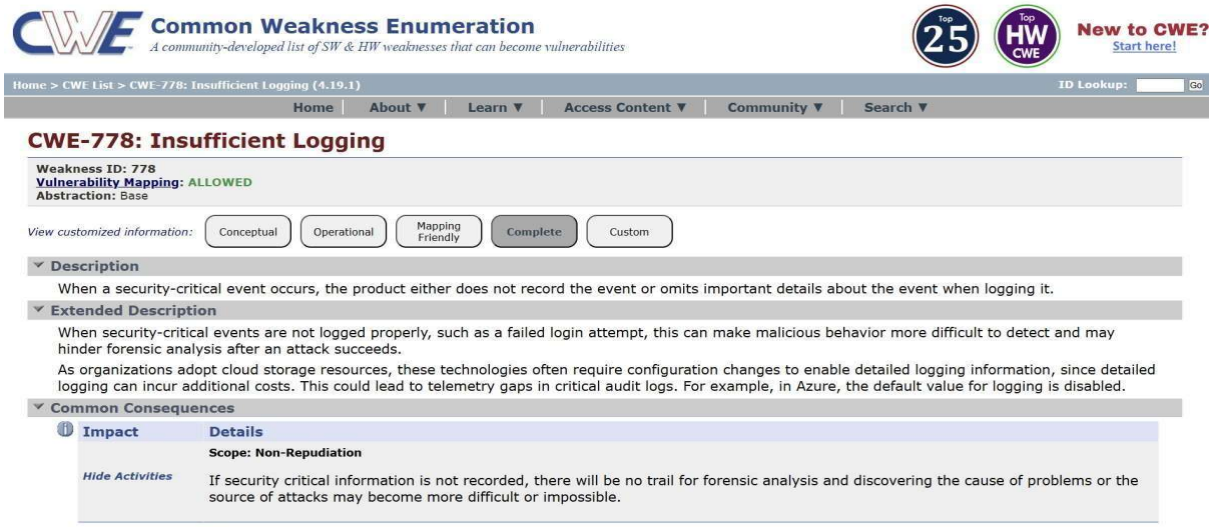
In many cloud-based and enterprise environments, logging features are sometimes disabled by default or require additional configuration. Organizations may also limit logging due to storage costs or performance concerns. However, these limitations can create gaps in security monitoring, leaving systems vulnerable to undetected attacks. Without adequate logs, security teams cannot perform effective forensic analysis or identify the source of security incidents.

Proper logging should capture detailed information about authentication events, system errors, configuration changes, user activities, and security alerts. Logs should also be securely stored, monitored regularly, and integrated with security monitoring systems such as SIEM tools to ensure early detection of threats.

Business Impact

The business impact of insufficient logging can be severe for organizations because it directly affects their ability to detect, investigate, and respond to cyber threats. When security events are not properly logged, attackers can perform malicious activities within the system without leaving any trace of their actions. This means organizations may not even realize that a security breach has occurred until significant damage has already been done.

Without proper logging, incident response teams cannot determine how an attack happened, which systems were affected, or what data may have been compromised. This lack of visibility makes forensic investigations extremely difficult and delays the organization's ability to contain and recover from security incidents. As a result, attackers may remain inside the network for extended periods of time, increasing the risk of data theft, system manipulation, or service disruption.



REPORT

9.Vulnerability Name

Inclusion of Web Functionality from an Untrusted Source

CWE / CVE CWE-830

OWASP / SANS Category Mapped to OWASP Foundation Top 10 – A08: Software and Data Integrity Failures

Description

The vulnerability occurs when a web application includes or executes external functionality, such as JavaScript files, widgets, plugins, or other resources, from a domain that is not fully trusted or verified. When these external components are loaded, they execute within the context of the hosting application's domain. This means the external code may gain the same privileges as the application itself.

In many modern web environments, developers include third-party functionality using HTML

<script> tags or external APIs to add features like analytics tracking, user interface enhancements, advertisements, or social media integrations. However, if the external domain is compromised, manipulated during transmission, or intentionally malicious, it can inject harmful code into the application.

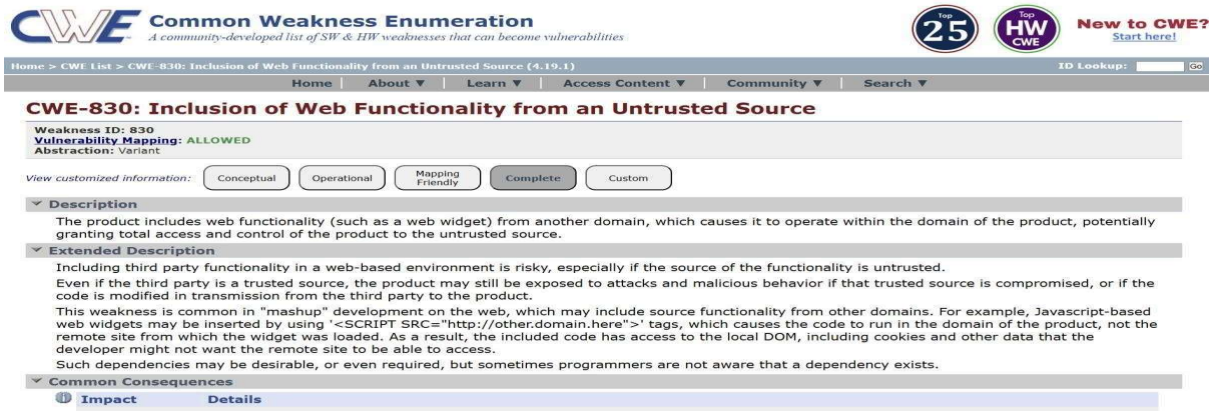
Because the browser executes the external code with the same permissions as local scripts, the malicious functionality can access sensitive information such as session tokens, cookies, or user input fields. It may also modify page content, redirect users to phishing sites, or perform unauthorized actions on behalf of the user. This vulnerability is especially common in mashup applications where multiple external sources are integrated without strict security controls.

Developers often overlook this risk because the external functionality appears legitimate at the time of integration. However, if the source later becomes compromised or the script is modified, all applications relying on it become vulnerable. Proper security mechanisms such as Content Security Policy (CSP), Subresource Integrity (SRI), and strict domain whitelisting are necessary to reduce the risks associated with this vulnerability.

The inclusion of web functionality from an untrusted source can have severe consequences for organizations, users, and business operations. If an attacker compromises the external resource or injects malicious code into a third-party script, the malicious code will automatically execute in every user's browser when they access the affected application. This can lead to large-scale data breaches, unauthorized access to sensitive user information, and manipulation of application functionality. Attackers may steal authentication cookies, enabling them to hijack user sessions and gain access to accounts without needing passwords.

Additionally, malicious scripts could modify web content to redirect users to phishing websites, distribute malware, or display fraudulent information. This not only compromises the security of users but also damages the organization's reputation and trustworthiness. Customers who experience security incidents may lose confidence in the organization's ability to protect their data, potentially resulting in financial losses, legal consequences, and regulatory penalties. In large-scale systems such as ecommerce platforms, banking services, or enterprise applications, this vulnerability could disrupt business operations and expose confidential corporate data.

Furthermore, organizations may face compliance violations if sensitive information such as personal data, financial records, or confidential business information is exposed through compromised external scripts. The cost of incident response, system recovery, legal liabilities, and reputation management can be extremely high. Therefore, organizations must implement strict security policies when integrating third-party



REPORT

10. Vulnerability Name

    Use of Default Credentials

CWE/CVE

The vulnerability identified in this assessment corresponds to CWE-1392: Use of Default Credentials. This weakness occurs when a product or system relies on predefined authentication credentials such as default usernames, passwords, or cryptographic keys for critical functionality. These credentials are typically embedded within the system to simplify the setup and deployment process. However, if administrators fail to modify these credentials after installation, attackers can easily exploit them to gain access to the system. Since many default credentials are publicly documented in product manuals or online databases, attackers often use automated tools to scan for systems that still use default login information. Once an attacker successfully logs in using these credentials, they may gain access to sensitive resources, administrative controls, or internal system configurations. This vulnerability highlights the importance of enforcing secure authentication practices and ensuring that all

default credentials are replaced with strong, unique passwords before a system is placed into production.

OWASP/SANS Category

This vulnerability is closely related to security issues described in the OWASP Top 10 2021, particularly A07: Identification and Authentication Failures. This category addresses weaknesses in authentication mechanisms that may allow attackers to bypass login controls or gain unauthorized access to systems and applications. The use of default credentials directly contributes to authentication failures because it allows attackers to log in using widely known or easily guessable passwords. Additionally, this issue aligns with the security risks highlighted in the SANS Top 25 Most Dangerous Software Errors, which identifies improper authentication and weak credential management as major sources of security vulnerabilities in modern software systems. Organizations that fail to enforce strong password policies or neglect to change default credentials during system deployment significantly increase the risk of unauthorized access and system compromise.

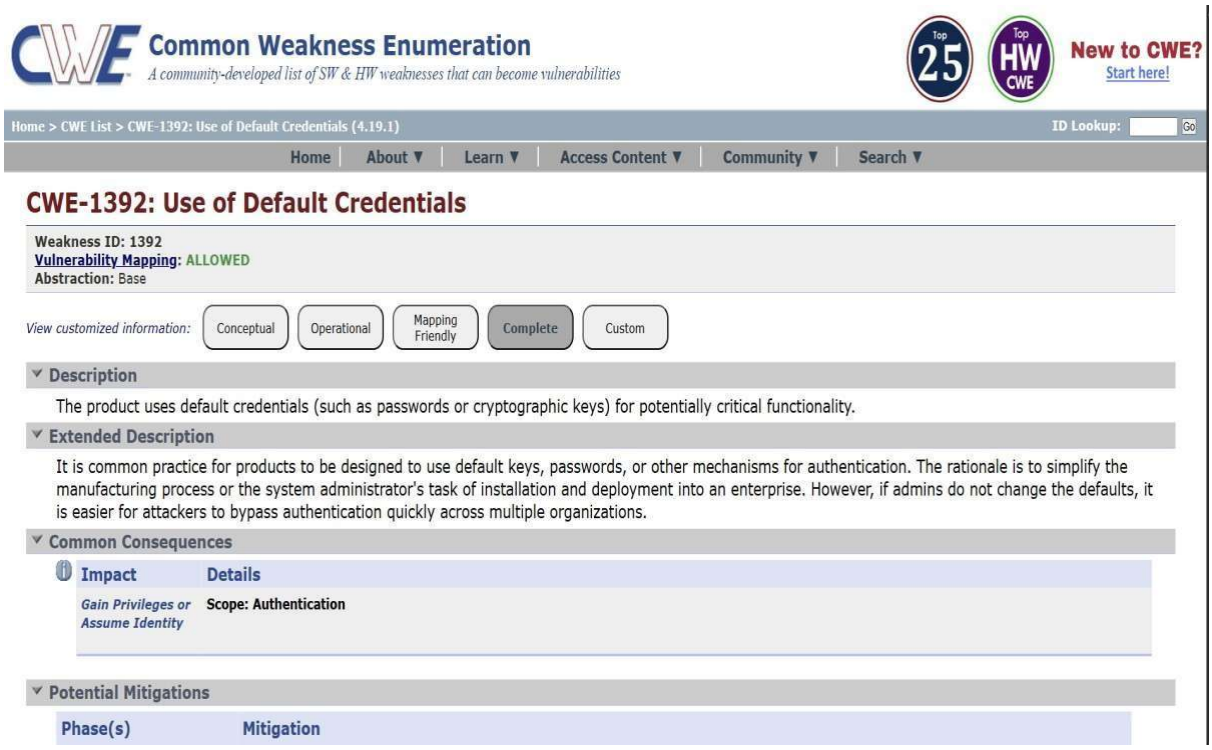
Description

Default credentials are commonly included in software products, network devices, and web applications to simplify the installation and configuration process for administrators and system integrators. These credentials typically consist of predefined usernames and passwords that allow administrators to access the system during the initial setup phase. While this practice may improve convenience during deployment, it also introduces significant security risks if the default credentials are not changed afterward. Attackers are well aware of common default credentials used by various systems and often maintain databases of these credentials for exploitation purposes. In many cases, automated scanning tools are used to identify systems that still rely on default login information. Once an attacker successfully logs in using default credentials, they may gain access to critical system functionality, sensitive data, or administrative controls. This access can allow them to modify system configurations, install malicious software, or extract confidential information from the system. The vulnerability becomes particularly dangerous in enterprise environments where systems are connected to internal networks and contain sensitive organizational data. Because default credentials are easily obtainable and widely known, their continued use in production

systems creates a serious security weakness that must be addressed through proper configuration management and secure authentication practices.

Business Impact

The use of default credentials within a system can lead to severe security and operational consequences for an organization. If attackers exploit this vulnerability, they may gain unauthorized access to sensitive systems, applications, or databases without needing sophisticated attack techniques. This unauthorized access may allow attackers to steal confidential information, manipulate system data, disrupt business operations, or deploy malware within the organization's network. In many cases, attackers who gain administrative access through default credentials can maintain persistent control over the system and launch additional attacks against other connected systems. Such security breaches can lead to financial losses, operational downtime, and potential legal liabilities for the affected organization. Additionally, organizations may face regulatory penalties if sensitive customer or business data is compromised due to poor security practices. Beyond the financial and legal consequences, security incidents resulting from default credential exploitation can significantly damage an organization's reputation and reduce customer trust. Clients and stakeholders expect organizations to maintain strong security controls to protect sensitive information, and failures in this area can result in long-term reputational harm. Therefore, addressing this vulnerability is essential to maintaining system integrity, protecting organizational assets, and ensuring compliance with industry security standards and best practices.



Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

Top 25 | Top HW CWE | New to CWE? Start here!

Home > CWE List > CWE-1392: Use of Default Credentials (4.19.1) | ID Lookup: Go

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-1392: Use of Default Credentials

Weakness ID: 1392
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product uses default credentials (such as passwords or cryptographic keys) for potentially critical functionality.

Extended Description

It is common practice for products to be designed to use default keys, passwords, or other mechanisms for authentication. The rationale is to simplify the manufacturing process or the system administrator's task of installation and deployment into an enterprise. However, if admins do not change the defaults, it is easier for attackers to bypass authentication quickly across multiple organizations.

Common Consequences

Impact	Details
Gain Privileges or Assume Identity	**Scope: Authentication**

Potential Mitigations

Phase(s)	Mitigation

STAGE 2

Overview :-

This report presents the results of a web application vulnerability assessment. The assessment identified several security weaknesses that could allow attackers to compromise the confidentiality, integrity, and availability of the system. The vulnerabilities mainly affect web application input validation, authentication mechanisms, configuration settings, and data protection practices. Addressing these issues will reduce the risk of data breaches, unauthorized access, and system compromise.

Target website

Not specified in the provided document.

Target ip address:-

Not specified in the provided document.

List of vulnerability:

S.No	Vulnerability Name	Severity	Port	Description
1	SQL Injection	High	80 / 443	Unsanitized user input allows attackers to manipulate SQL queries and access or modify database data.
2	Cross-Site Scripting (XSS)	High	80 / 443	Attackers inject malicious scripts into web pages that execute in users' browsers.
3	Insecure Direct Object References (IDOR)	Medium	80 / 443	Users can access unauthorized data by modifying object identifiers in requests.
4	Directory Traversal	Medium	80	Attackers manipulate file paths to access restricted files or directories.
5	Broken Authentication	High	443	Weak authentication and session management allow attackers to impersonate users.
6	Security Misconfiguration	Low	443	Default settings or improper configurations expose the system to attacks.

7	Cross-Site Request Forgery (CSRF)	Low	8080 / 443	Attackers trick authenticated users into performing unwanted actions.
8	Unvalidated Redirects and Forwards	Medium	80 / 443	Users can be redirected to malicious websites through manipulated URLs.
9	Sensitive Data Exposure	High	443	Sensitive information such as passwords or financial data may be exposed due to weak protection mechanisms.

Report

The vulnerability scan identified multiple high, medium, and low severity vulnerabilities in the web application. High-risk issues such as SQL Injection, Cross-Site Scripting, Broken Authentication, and Sensitive Data Exposure may allow attackers to access or manipulate sensitive data, hijack sessions, or take control of user accounts. Medium-severity issues like IDOR, Directory Traversal, and Unvalidated Redirects could allow unauthorized data access or phishing attacks. Low-severity issues such as Security Misconfiguration and CSRF increase the attack surface and may assist attackers in exploiting other vulnerabilities. Immediate remediation steps such as implementing secure coding practices, proper input validation, strong authentication controls, and secure configuration management are recommended.

1. VULNERABILITY NAME:

SEVERITY: High

PLUGIN: Input Validation Failure

PORT: 80/443

DESCRIPTION: Unsanitized user input in query parameters allows attackers to manipulate SQL queries, potentially exposing or altering sensitive data.

SOLUTION:Use parameterized queries and stored procedures.

Employ ORM frameworks with built-in protections.

Regularly test with automated scanners.

BUSINESS IMPACT: Data theft, unauthorized account access, and reputational damage.

2. VULNERABILITY NAME: Cross-Site Scripting (XSS)

SEVERITY: High

PLUGIN: Reflected/Stored XSS Detection

PORT: 80/443

DESCRIPTION: Malicious scripts can be injected into web pages viewed by other users.

SOLUTION:Encode output properly.

Use Content Security Policy (CSP).

Sanitize all user inputs.

BUSINESS IMPACT: Session hijacking, defacement, and loss of customer trust.

3. VULNERABILITY NAME: Insecure Direct Object References (IDOR)

SEVERITY: Medium

PLUGIN: Authorization Bypass Detection

PORT: 80/443

DESCRIPTION: Users can access objects by manipulating identifiers in requests without proper authorization checks.

SOLUTION:Enforce access control checks at every layer.

Use indirect references instead of exposing raw IDs.

BUSINESS IMPACT: Unauthorized data exposure and compliance violations.

4. VULNERABILITY NAME: Directory Traversal

SEVERITY: Medium

PLUGIN: Path Manipulation Detection

PORT: 80

DESCRIPTION: Attackers can manipulate file paths to access restricted directories or files.

SOLUTION:Normalize and validate input paths.

Restrict file system access.

BUSINESS IMPACT: Leakage of sensitive configuration files.

5. VULNERABILITY NAME: Broken Authentication

SEVERITY: High

PLUGIN: Weak Session Management

PORT: 443

DESCRIPTION: Poorly implemented authentication mechanisms allow attackers to impersonate users.

SOLUTION:Enforce MFA.

Use secure session tokens with expiration.

BUSINESS IMPACT: Account takeover and fraud.

6. VULNERABILITY NAME: Security Misconfiguration

SEVERITY: Low

PLUGIN: Default Credentials/Weak Configurations

PORT: 443

DESCRIPTION: Default settings or unnecessary services expose systems to attack.

SOLUTION:Harden configurations.

Remove unused services.

BUSINESS IMPACT: Increased attack surface.

7. VULNERABILITY NAME: Cross-Site Request Forgery (CSRF)

SEVERITY: Low

PLUGIN: HTTP Form-Based Authentication Detected

PORT: 8080/443

DESCRIPTION: Attackers trick authenticated users into executing unwanted actions.

SOLUTION:Implement anti-CSRF tokens.

Use SameSite cookie attributes.

BUSINESS IMPACT: Unauthorized transactions and compromised accounts.

9. VULNERABILITY NAME: Unvalidated Redirects and Forwards

SEVERITY: Medium

PLUGIN: URL Redirection Detection

PORT: 80/443

DESCRIPTION: Attackers can redirect users to malicious sites via unvalidated parameters.

SOLUTION:Validate and whitelist redirect targets.

Avoid using user-supplied input for redirects.

BUSINESS IMPACT: Phishing and malware distribution.

10. VULNERABILITY NAME: Sensitive Data Exposure

SEVERITY: High

PLUGIN: Information Disclosure Detection

PORT: 443

DESCRIPTION: Improper handling of sensitive data (e.g., passwords, credit card numbers) exposes it to attackers.

SOLUTION:Encrypt data at rest and in transit.

Mask sensitive fields in logs.

BUSINESS IMPACT: Identity theft, financial loss, and legal penalties.

STAGE 3

Title:

Analysis of Security Monitoring Capabilities Using SOC, SIEM and QRadar

SOC

A Security Operations Center (SOC) is a centralized team responsible for continuously monitoring, detecting, analyzing, and responding to cybersecurity incidents within an organization. The SOC team uses various security tools, threat intelligence, and monitoring platforms to identify suspicious activities in the network. Their main objective is to protect organizational systems, data, and infrastructure from cyber threats such as malware, phishing attacks, data breaches, and unauthorized access. SOC teams work 24/7 to ensure that any potential security incident is quickly detected and mitigated before it causes serious damage to the organization.

SOC Cycle

The SOC cycle represents the continuous process followed by the Security Operations Center to manage cybersecurity threats. It typically includes stages such as monitoring, detection, investigation, response, and recovery. First, the SOC continuously monitors network activities and system logs using security tools. When suspicious behavior is detected, analysts investigate the incident to determine whether it is a real threat. If confirmed, appropriate response actions such as blocking malicious IP addresses or isolating compromised systems are taken. Finally, the SOC performs recovery actions and improves security controls to prevent similar attacks in the future.

SIEM

Security Information and Event Management (SIEM) is a security solution that collects, analyzes, and correlates security logs and events from multiple sources within an organization's IT environment. SIEM tools help security teams identify unusual patterns or malicious activities by analyzing large amounts of log data in real time. These tools provide centralized monitoring of networks, servers, applications, and devices. SIEM solutions are widely used in SOC environments because they allow security teams to detect threats quickly and respond effectively to potential cyber attacks.

SIEM Cycle

The SIEM cycle involves several important stages that ensure effective security monitoring and threat detection. The first stage is data collection, where logs and events are gathered from various sources such as firewalls, servers, network devices, and applications. The second stage is normalization and correlation, where the system analyzes and compares events to identify suspicious patterns. The third stage is alert generation, where SIEM tools notify security analysts about potential threats. Finally, security teams investigate alerts and respond to incidents by taking necessary actions to mitigate the risk.

MISP

MISP (Malware Information Sharing Platform) is an open-source threat intelligence platform designed for sharing information about cybersecurity threats. It allows organizations to exchange information related to malware indicators, attack techniques, malicious IP addresses,

and threat actors. By sharing threat intelligence, organizations can better understand emerging threats and improve their security defenses. MISP helps security teams collaborate and detect cyber attacks faster by providing a centralized platform for threat intelligence analysis and sharing.

.

Threat Intelligence

Threat intelligence refers to the collection and analysis of information about potential or existing cyber threats. It helps organizations understand attacker behavior, attack techniques, and emerging vulnerabilities. Threat intelligence data may include malicious IP addresses, malware signatures, phishing domains, and hacker group activities. Security teams use this information to improve their defensive strategies and prevent attacks before they occur. Integrating threat intelligence with SIEM systems allows organizations to identify threats more effectively and respond quickly to potential security incidents.

Your College Network Information

The network infrastructure of A.V.P College of Arts and Science for Women consists of multiple interconnected systems such as administrative office computers, computer laboratories used by students, faculty systems, servers, Wi-Fi networks, and internet gateways. These networks support several academic and administrative activities including online learning platforms, digital library access, email communication, examination management systems, and institutional databases. Since a large number of students, faculty members, and staff access the network daily, maintaining strong cybersecurity measures is essential. Security controls such as firewalls, antivirus systems, access control mechanisms, and network monitoring tools help protect sensitive academic records, student information, and administrative data from cyber threats such as malware attacks, unauthorized access, and data breaches.

How You Think You Deploy SOC in Your College

To enhance cybersecurity in A.V.P College of Arts and Science for Women, a Security Operations Center (SOC) could be implemented to monitor and manage the college network infrastructure. The SOC would collect logs and security events from various devices such as servers, routers, firewalls, and endpoint computers using a SIEM platform. Security analysts within the SOC would continuously monitor these logs to identify suspicious activities like

repeated login failures, unusual network traffic, or unauthorized access attempts. Additionally, the SOC could integrate threat intelligence sources, automated alert systems, and incident response procedures to quickly detect and respond to potential cyber threats. Implementing a SOC would significantly improve the college's ability to safeguard student data, academic systems, and institutional resources from cyber attacks.

Incident Response

Incident response is the process of identifying, managing, and resolving cybersecurity incidents within an organization. When a security breach or suspicious activity is detected, incident response teams follow structured procedures to contain the threat and minimize damage. The process usually includes identification, containment, eradication, recovery, and post-incident analysis. Effective incident response helps organizations restore normal operations quickly and prevent similar attacks in the future by improving their security defenses.

QRadar & Understanding About Tool

IBM QRadar is a widely used SIEM platform that helps organizations monitor security events and detect cyber threats in real time. QRadar collects logs and network flow data from multiple devices across the organization and analyzes them to identify suspicious activities. The tool uses advanced analytics and correlation rules to generate alerts for potential security incidents. Security analysts use the QRadar dashboard to investigate threats, track attack patterns, and respond to incidents efficiently. By providing centralized security visibility, QRadar plays an important role in modern SOC environments.

Conclusion

Stage 1 – What I understood from Web Application Testing

From Stage 1, I understood the importance of identifying vulnerabilities in software applications. Web application testing helps security professionals detect weaknesses such as SQL Injection, Cross-Site Scripting, and authentication issues. By understanding CWE standards and common vulnerabilities, developers can implement secure coding practices and reduce security risks in applications.

Stage 2 – What I understood from the Nessus Report

In Stage 2, the vulnerability assessment report showed how automated security scanning tools can identify weaknesses in web applications. The report highlighted vulnerabilities such as SQL Injection, Broken Authentication, and Sensitive Data Exposure, which can lead to serious security breaches if not addressed. It also demonstrated the importance of vulnerability management and proper security configurations in protecting systems from cyber attacks.

Stage 3 – What I understood from SOC / SIEM / QRadar Dashboard

From Stage 3, I learned how SOC teams use SIEM tools like QRadar to monitor network activity and detect potential threats. The dashboard provides real-time visibility into security events and helps analysts investigate suspicious activities. This stage highlighted the importance of centralized monitoring, threat intelligence integration, and incident response in modern cybersecurity operations.

Future Scope

Stage 1 – Future Scope of Web Application Testing

The future of web application testing will involve more advanced automated security tools, AIbased vulnerability detection, and secure development practices. Continuous security testing during the development lifecycle will help organizations identify vulnerabilities earlier and reduce the risk of cyber attacks.

Stage 2 – Future Scope of Testing Process

The vulnerability testing process will continue to improve with advanced scanning technologies, automated risk assessment tools, and integration with DevSecOps environments. Organizations will adopt continuous vulnerability monitoring to ensure that systems remain secure against evolving cyber threats.

Stage 3 – Future Scope of SOC / SIEM

The future of SOC and SIEM technologies will include artificial intelligence, machine learning, and automated threat detection systems. These technologies will help security teams detect attacks faster and respond to incidents more efficiently. Cloud-based SIEM platforms and automated security orchestration tools will further improve cybersecurity operations.

Title of the Project:

Analysis and Impact Assessment of Common Software Security Vulnerabilities Using CWE Standards

Overview:

In today's digital world, software applications are widely used in industries such as banking, healthcare, education, government systems, and e-commerce. As the dependency on software systems increases, security vulnerabilities also become a major concern. Cyber attackers continuously search for weaknesses in applications to exploit them for unauthorized access, data theft, or service disruption. Therefore, identifying and understanding common software vulnerabilities is essential for developers, security professionals, and organizations.

This project focuses on analyzing common software vulnerabilities based on the Common Weakness Enumeration (CWE) standard. CWE is a well-known classification system developed to identify and categorize weaknesses in software and hardware. These weaknesses can potentially lead to serious security issues if they are not properly addressed during software development or deployment. By studying these vulnerabilities, developers can understand the root causes of security problems and implement proper mitigation techniques.

The primary objective of this project is to study ten common vulnerabilities that frequently appear in modern applications. Each vulnerability will be analyzed in terms of its CWE identification number, description, security category according to OWASP or SANS, and its potential business impact. Understanding the business impact is important because vulnerabilities do not only affect technical systems but also influence financial stability, organizational reputation, customer trust, and legal compliance.

This project will examine vulnerabilities such as Null Pointer Dereference, Sensitive Information Exposure in Logs, Reliance on Cookies without Validation, Buffer Overflow, SQL Injection, Cross-Site Scripting, Improper Input Validation, Broken Authentication, Insecure Direct Object References, and Missing Authorization Checks. These vulnerabilities are commonly found in web applications, mobile applications, and enterprise software systems.

Another important goal of this project is to demonstrate how poor coding practices, lack of input validation, improper error handling, and insecure authentication mechanisms can create serious security risks. By documenting these weaknesses, developers and security teams can better understand how attackers exploit these vulnerabilities and how proper secure coding practices can prevent them.

This project also highlights the relationship between CWE vulnerabilities and OWASP/SANS security categories, which are widely used frameworks in cybersecurity. OWASP provides the well-known OWASP Top 10 list, which describes the most critical security risks in web applications. By mapping vulnerabilities to these categories, the project provides a clearer understanding of how real-world attacks occur.

In conclusion, this project provides a comprehensive overview of common software vulnerabilities and their impact on organizations. By analyzing different CWE vulnerabilities and their business implications, this project aims to raise awareness about secure software development practices and the importance of implementing strong security controls in modern applications.

List of teammates:

S.no	name	collage	contact
1	Deepika V	AVP College of Arts and Science	deepikavenkatesan121@gmail.com
2	Charumathi A	AVP College of Arts and Science	Charumathiarumugam12@gmail.com
3	Priya dharshini B	AVP College of Arts and Science	Priyadharshinibakiyaraj906@gmail.com
4	Moulisha S	AVP College of Arts and Science	
5	Harshini N	AVP College of Arts and Science	

List of Vulnerability Table

S.no	Vulnerability Name	CWE/CVE No

1	NULL Pointer Dereference	CWE-476
2	Insertion of Sensitive Information into Log File	CWE-532
3	Reliance on Cookies without Validation	CWE-784
4	Buffer Overflow	CWE-120
5	SQL Injection	CWE-89
6	Cross-Site Scripting (XSS)	CWE-79
7	Improper Input Validation	CWE-20
8	Broken Authentication	CWE-287
9	Insecure Direct Object Reference	CWE-639
10	Missing Authorization Check	CWE-862

REPORT

1.Vulnerability Name

Sensitive Information Disclosure in Source Code Comments

CWE / CVE

The identified vulnerability is classified as CWE-615, which is maintained by the MITRE Corporation.

This classification describes a security weakness where developers leave confidential information inside comments within the source code. These comments may expose internal details about the application's design or implementation that should not be accessible to users or attackers.

OWASP / SANS Category

According to the OWASP Foundation, this issue is commonly associated with Sensitive Data Exposure and sometimes related to Security Misconfiguration in the OWASP Top 10 security risks.

These categories highlight the importance of protecting confidential information within applications and ensuring that internal implementation details are not unnecessarily exposed

Description

Sensitive Information in Source Code Comments is a security weakness that arises when developers leave confidential or internal information within comments embedded in the application's codebase. Comments are commonly used to document code behavior, provide instructions to developers, or explain complex algorithms. While this practice improves readability and collaboration during development, it may also unintentionally reveal information that attackers can exploit.

For example, developers may leave comments containing internal file structures, test URLs, API endpoints, database schema details, or debugging instructions that were intended only for development purposes. If the application exposes its client-side source code or scripts to users, these comments can be easily viewed through browser developer tools or by inspecting downloaded files.

Attackers frequently examine source code comments because they may contain clues about hidden functionality or unfinished features within the system. By analyzing these comments, attackers can map out the internal architecture of the application, identify administrative interfaces, and determine potential entry points for further attacks.

In some cases, comments may reveal outdated code fragments or references to deprecated features that still exist in the system. These fragments may point attackers toward vulnerable components that were not properly removed from the application. As a result, even though comments themselves do not execute code, they can indirectly contribute to security breaches by exposing information that assists attackers during the reconnaissance phase of an attack.

Business Impact

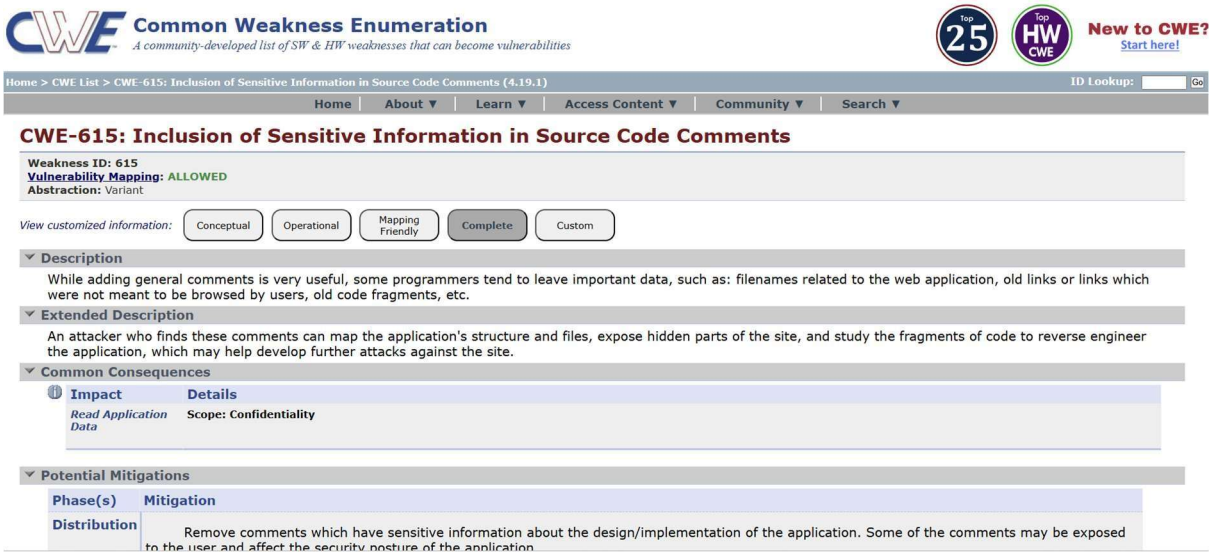
The presence of sensitive information within source code comments can significantly increase the risk of security breaches for an organization. Although comments are intended only for developers, their exposure in publicly accessible code may reveal important technical details about the internal structure of the application. Attackers can use this information to better understand how the system operates, which components interact with each other, and where potential weaknesses might exist. By analyzing these internal details, attackers can perform more targeted reconnaissance and reduce the time required to discover exploitable vulnerabilities within the system.

Another major concern is that comments may disclose hidden resources or internal endpoints that were never meant to be accessed by external users. For instance, comments might reference administrative login pages, testing environments, development server addresses, or temporary APIs used during the development process. If attackers discover these hidden paths, they may attempt to access them directly, potentially bypassing security controls or authentication mechanisms. This can lead to unauthorized access to sensitive data, modification of application functionality, or disruption of normal system operations.

In addition to technical risks, the exposure of sensitive information through comments can have serious business consequences. If attackers successfully exploit the information revealed in comments, the organization may suffer data breaches involving customer information, financial records, or proprietary business data. Such incidents can result in financial losses, legal liabilities, regulatory penalties, and reputational damage. Customers and stakeholders may lose trust in the organization's ability to protect their data, which can affect long-term business relationships and brand reputation.

Furthermore, organizations that fail to follow secure coding practices may face compliance issues with security standards and regulations. Many cybersecurity frameworks require organizations to implement strict controls over sensitive information and ensure that internal system details are not publicly exposed. Leaving sensitive data in source code comments may violate these requirements and lead to audit failures or compliance penalties.

For these reasons, it is essential for development teams to implement secure coding practices that prevent the inclusion of sensitive information in source code comments. Regular code reviews, automated security scans, and proper development guidelines can help ensure that confidential details are removed before applications are deployed to production environments. By addressing this issue proactively, organizations can reduce the likelihood of information leakage and strengthen the overall security of their software systems.



Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-615: Inclusion of Sensitive Information in Source Code Comments (4.19.1)

ID Lookup: Go

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-615: Inclusion of Sensitive Information in Source Code Comments

Weakness ID: 615
Vulnerability Mapping: ALLOWED
Abstraction: Variant

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

While adding general comments is very useful, some programmers tend to leave important data, such as: filenames related to the web application, old links or links which were not meant to be browsed by users, old code fragments, etc.

Extended Description

An attacker who finds these comments can map the application's structure and files, expose hidden parts of the site, and study the fragments of code to reverse engineer the application, which may help develop further attacks against the site.

Common Consequences

Impact	Details
Read Application Data	Scope: Confidentiality

Potential Mitigations

Phase(s)	Mitigation
Distribution	Remove comments which have sensitive information about the design/implementation of the application. Some of the comments may be exposed to the user and affect the security posture of the application

REPORT

2.Vulnerability Name

Missing Authentication for Critical Function

CWE / CVE

The vulnerability is categorized as CWE-306, which is maintained by the MITRE Corporation. This weakness occurs when an application fails to verify the identity of users before allowing them to access sensitive or critical system operations.

OWASP / SANS Category

According to the OWASP Foundation, this vulnerability is closely related to Identification and Authentication Failures, one of the major risks listed in the OWASP Top 10 security risks for modern web applications.

Description

The vulnerability known as Missing Authentication for Critical Function occurs when an application allows users to access sensitive or important system operations without verifying their identity. Authentication mechanisms are designed to ensure that only authorized users can perform certain actions within a system. When these mechanisms are missing or improperly implemented, attackers may be able to access restricted features without providing valid credentials.

In many web applications, certain functions are considered critical because they allow users to perform actions that can significantly impact the system. Examples include modifying user accounts, accessing confidential data, updating application settings, managing financial transactions, or controlling administrative features. These functions must always be protected by authentication and authorization checks to ensure that only legitimate users can perform them.

However, developers sometimes overlook authentication requirements for certain endpoints or internal APIs. This may occur when developers assume that the function will only be accessed by trusted users or internal systems. In reality, attackers can discover these unprotected endpoints through various techniques such as inspecting web traffic, reviewing client-side scripts, or performing automated vulnerability scans.

Once attackers identify an endpoint that lacks authentication, they may exploit it to perform unauthorized actions. For example, an attacker could access administrative functionality without logging in, modify application configurations, retrieve sensitive information, or manipulate data stored within the system. This vulnerability can therefore serve as a gateway for more severe attacks, especially when combined with other weaknesses in the application.

Furthermore, missing authentication controls can allow automated bots or malicious users to repeatedly access system resources, potentially leading to system misuse, data corruption, or

service disruption. For these reasons, this vulnerability is considered a serious security risk and must be addressed during the development and deployment phases of the application lifecycle.

Business Impact

The absence of proper authentication mechanisms for critical system functions can have severe consequences for an organization's security and operational stability. When sensitive operations are accessible without verifying user identity, attackers may gain unauthorized control over important components of the system. This can lead to unauthorized data access, modification of system configurations, and disruption of normal business operations. In environments where applications handle confidential information such as customer data, financial records, or proprietary business information, the consequences of such unauthorized access can be particularly damaging.

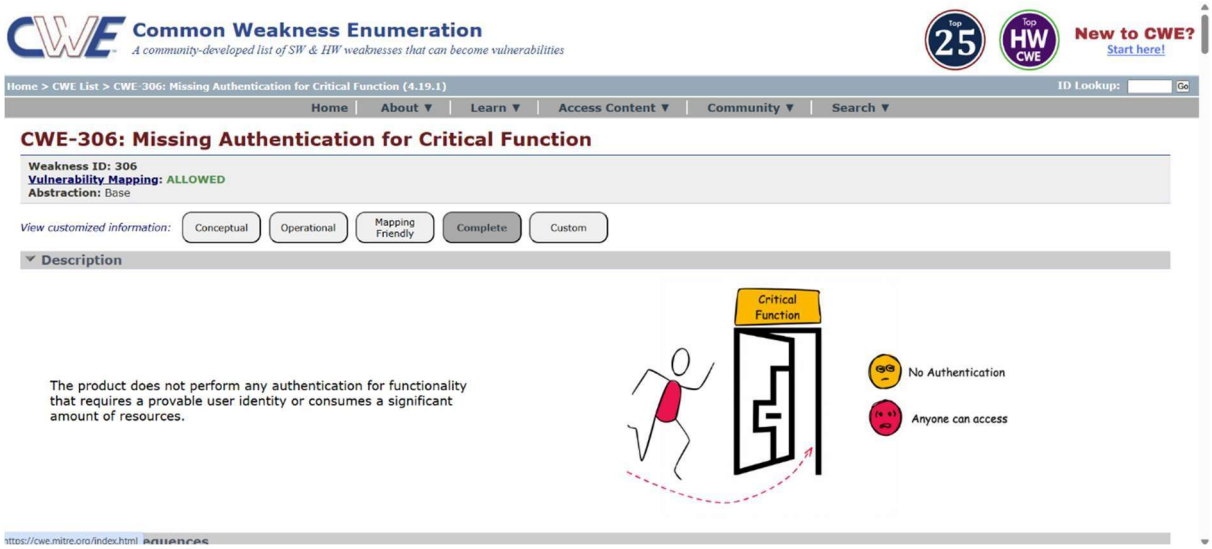
Another major concern is that attackers can exploit these unprotected functions to escalate their privileges within the system. For example, if administrative features are accessible without authentication, an attacker may be able to create new user accounts, modify existing permissions, or disable security controls. Such actions can compromise the integrity of the entire application environment and allow attackers to maintain persistent access to the system.

From a financial perspective, organizations may suffer significant losses if attackers exploit authentication vulnerabilities. Unauthorized access to financial systems can lead to fraudulent transactions, theft of sensitive data, or manipulation of critical records. In addition, organizations may incur costs related to incident response, system recovery, and implementation of additional security controls after a breach has occurred.

Beyond financial damage, organizations may also experience reputational harm and loss of customer trust. Customers expect companies to protect their personal information and maintain secure systems. If a security breach occurs due to weak authentication practices, it may negatively affect the organization's public image and lead to decreased customer confidence. In some industries, regulatory bodies also require organizations to implement strict

authentication controls to protect sensitive data, and failure to comply with these regulations may result in legal penalties or compliance violations.

Therefore, ensuring that all critical system functions are protected by strong authentication mechanisms is essential for maintaining the security, reliability, and trustworthiness of modern web applications. Organizations must adopt secure development practices, conduct regular security assessments, and implement robust authentication frameworks to prevent unauthorized access to sensitive operations.



REPORT

3.Vulnerability Name

Reliance on Cookies without Validation and Integrity Checking

CWE / CVE

The vulnerability is categorized as CWE-784, which is maintained by the MITRE Corporation. This weakness occurs when applications rely on cookie values to make security decisions without verifying the integrity or authenticity of the data contained within the cookie.

OWASP / SANS Category

According to the OWASP Foundation, this vulnerability is closely related to Identification and Authentication Failures and improper session management practices in web applications.

Description

The vulnerability known as Reliance on Cookies without Validation and Integrity Checking occurs when an application trusts data stored in browser cookies without verifying whether that data has been modified by the user. Cookies are stored on the client side, meaning they reside in the user's browser rather than on the server. Because of this, attackers can easily access and modify cookie values using browser developer tools or specialized tools designed for manipulating web traffic.

In many web applications, cookies are used to store important information such as session identifiers, user roles, authentication status, or preferences. If the application uses these cookie values to determine whether a user is authenticated or authorized to access certain features, attackers may attempt to manipulate them to gain unauthorized privileges.

For example, an application might store a value in a cookie indicating that the user is an administrator. If this cookie value is not properly validated or protected using cryptographic techniques, an attacker could manually modify the cookie to change their role from a normal user to an administrator. Once this modification is accepted by the server, the attacker may gain access to restricted functionality such as administrative dashboards, configuration settings, or sensitive data.

Attackers can also intercept cookies during communication if proper transport security mechanisms such as HTTPS are not implemented. In such cases, attackers could steal valid session cookies and impersonate legitimate users. This type of attack, commonly known as session hijacking, can allow attackers to bypass authentication mechanisms entirely.

Furthermore, relying on unvalidated cookies for security decisions undermines the trust model of the application. Since cookies originate from the client side, they should never be trusted without verification. Secure applications typically store sensitive authentication and authorization data on the server and use cookies only as references to server-side session information. Without such safeguards, attackers can manipulate client-side data to bypass security restrictions and gain unauthorized access.

Business Impact

The misuse of cookies for making security decisions without proper validation can expose organizations to significant cybersecurity risks. When applications rely on client-controlled data such as cookies to determine user privileges or authentication status, attackers may manipulate these values to gain unauthorized access to sensitive resources. This can allow attackers to bypass authentication controls, impersonate legitimate users, or escalate their privileges within the system.

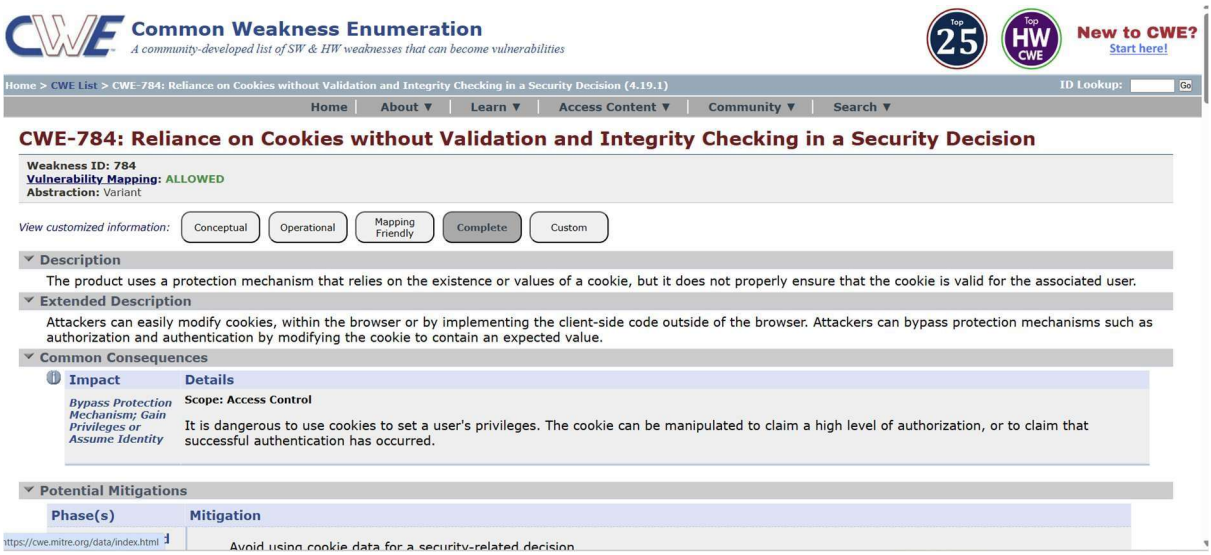
One of the most serious consequences of this vulnerability is unauthorized access to restricted areas of the application. If attackers can modify cookies to elevate their privileges, they may gain access to administrative interfaces or sensitive system functions. These interfaces often provide the ability to manage user accounts, modify system configurations, or access confidential data. Unauthorized access to such features can compromise the integrity and confidentiality of the entire system.

Another major risk associated with this vulnerability is session hijacking. If cookies are not properly secured or validated, attackers may intercept or steal session cookies and use them to impersonate legitimate users. Once attackers gain access to a valid session, they may perform actions on behalf of the victim without needing to know their login credentials. This can lead to unauthorized transactions, data theft, or manipulation of user accounts.

From a business perspective, the consequences of cookie manipulation attacks can be severe. Organizations may suffer financial losses due to fraud, data breaches, or operational disruptions. In addition, compromised systems may expose sensitive customer information such as personal data, financial records, or login credentials. This can lead to legal liabilities, regulatory penalties, and significant damage to the organization's reputation.

Loss of customer trust is another critical concern. Users expect organizations to protect their personal data and maintain secure systems. If a vulnerability allows attackers to bypass authentication controls or manipulate session data, customers may lose confidence in the organization's ability to safeguard their information. This can result in reduced customer engagement, negative publicity, and long-term reputational harm.

For these reasons, organizations must implement secure session management practices and ensure that cookie data cannot be trusted without proper validation. By enforcing strong security controls and validating all client-side data, organizations can reduce the risk of unauthorized access and protect both their systems and their users from potential cyber threats.



REPORT

4.Vulnerability Name

Sensitive Information Exposure in Log Files

CWE / CVE

The identified vulnerability is categorized as CWE-532, which is maintained by the MITRE Corporation.

This weakness describes a situation where sensitive or confidential data is written into application log files, making it possible for unauthorized individuals to access and misuse the information contained in those logs.

OWASP / SANS Category

According to the OWASP Foundation, this vulnerability is related to Sensitive Data Exposure and improper security logging practices in web applications.

Description

The vulnerability known as Insertion of Sensitive Information into Log File occurs when an application records confidential data in log files during normal operation. Logging systems are designed to capture information about application events such as user login attempts, error messages, system activity, and performance monitoring. However, if sensitive data is logged without proper consideration, it may unintentionally expose confidential information to unauthorized parties.

Sensitive information that may appear in log files includes user passwords, authentication tokens, credit card numbers, session identifiers, personal user data, or system configuration details. This data may be logged during authentication processes, error handling routines, debugging operations, or transaction monitoring activities. If the application records this information in plain text within log files, it becomes vulnerable to unauthorized access.

Attackers who gain access to log files can analyze them to extract sensitive information about users or system operations. For example, if passwords or session tokens are recorded in logs, attackers may use them to impersonate legitimate users or gain unauthorized access to the system. Similarly, log files may reveal internal system structures, database queries, or API endpoints that could assist attackers in identifying additional vulnerabilities.

Another risk arises when logs are shared across different systems or transmitted to centralized monitoring platforms. If log data is transmitted without encryption or stored in insecure environments, sensitive information may be intercepted or accessed by unauthorized individuals. Additionally, logs often remain stored for extended periods of time, increasing the likelihood that sensitive information may be exposed if the system is compromised.

Therefore, logging practices must be carefully designed to ensure that only necessary information is recorded while protecting confidential data from exposure. Sensitive fields should be masked, encrypted, or completely excluded from log entries to prevent attackers from exploiting log data.

Business Impact

The presence of sensitive information within log files can create serious security risks for organizations. Log files often contain detailed records of system activity, and if these logs include confidential information such as passwords, session tokens, or personal user data, attackers may be able to exploit them to gain unauthorized access to systems or compromise user accounts. Because logs are frequently stored for long periods of time and may be accessible to multiple systems or administrators, they represent a valuable target for attackers seeking to extract sensitive information.

One major consequence of this vulnerability is the potential exposure of user credentials or authentication tokens. If login attempts or authentication processes are logged without proper sanitization, attackers who gain access to these logs may retrieve valid credentials and use them to impersonate legitimate users. This could allow attackers to bypass authentication mechanisms and access sensitive areas of the application without detection. Such unauthorized access may result in data theft, manipulation of records, or disruption of business operations.

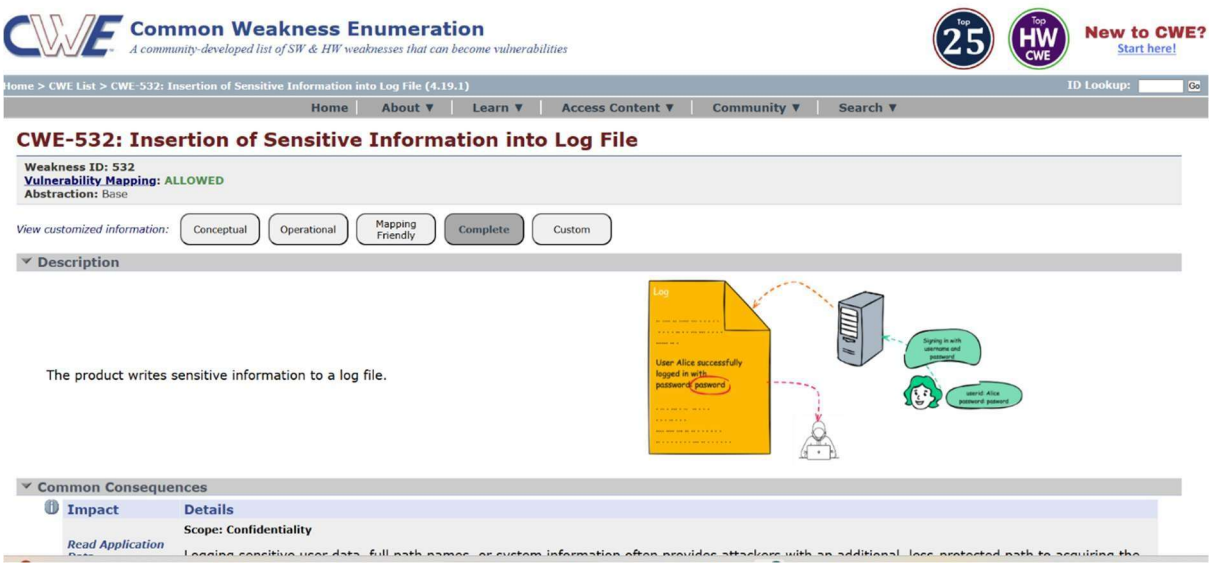
Another significant risk involves the leakage of personal or financial information. Applications that process sensitive customer data may inadvertently record details such as personal identification information, financial transactions, or payment data within log files. If these logs are exposed, attackers could collect large amounts of sensitive user information, potentially leading to identity theft, financial fraud, or other malicious activities. In addition to harming affected users, such incidents can severely damage the reputation of the organization responsible for the data.

Organizations may also face regulatory and legal consequences if sensitive data is improperly stored in log files. Many data protection regulations require companies to implement strict safeguards for handling personal and financial information. Failure to protect this data can result in compliance violations, legal penalties, and financial liabilities. Regulatory

investigations following a data breach may also require organizations to invest significant resources in remediation and security improvements.

Beyond financial and legal consequences, the exposure of sensitive information through log files can damage customer trust and public reputation. Customers expect organizations to protect their data and implement strong security practices. If sensitive information is leaked due to poor logging practices, customers may lose confidence in the organization's ability to safeguard their personal information. This loss of trust can have long-term effects on customer relationships, brand reputation, and business growth.

For these reasons, organizations must implement secure logging practices and ensure that sensitive information is never recorded in logs unless absolutely necessary. By carefully controlling what data is logged and implementing proper access controls, encryption, and monitoring mechanisms, organizations can reduce the risk of information leakage and protect both their systems and their users from potential cyber threats.



REPORT

5.Vulnerability Name

NULL Pointer Dereference

CWE / CVE

This vulnerability is classified as CWE-476 in the Common Weakness Enumeration system maintained by the MITRE Corporation.

CWE-476 refers to situations where a software application attempts to dereference a pointer that has not been properly initialized or has been assigned a NULL value. Because NULL represents an invalid or empty memory reference, attempting to access it results in program failure or system crashes. OASP / SANS Category

According to secure development guidance from the OWASP Foundation, NULL pointer dereference issues fall under software implementation vulnerabilities that arise from improper input validation, insufficient error handling, or unsafe memory management practices.

These vulnerabilities are typically associated with secure coding weaknesses that can affect application availability and reliability.

Description

The NULL Pointer Dereference vulnerability occurs when a program attempts to access memory through a pointer that is set to NULL instead of referencing a valid memory address. In programming, a pointer is used to store the memory location of a variable or object. If the pointer does not point to a valid location but the program attempts to use it, the system cannot retrieve or modify the data as expected.

This situation often arises when memory allocation fails or when a pointer variable is declared but not properly initialized. Developers may also inadvertently assign NULL values to pointers during program execution without implementing validation checks before accessing the pointer. When the program later attempts to dereference the pointer, the operating system detects that the pointer refers to an invalid memory location and terminates the process.

In many software environments, dereferencing a NULL pointer causes an application crash. This crash may interrupt normal system operations and affect users who depend on the software for critical tasks. In server environments or web applications, repeated crashes caused by this vulnerability can disrupt services and reduce system availability.

Although NULL pointer dereference vulnerabilities are often considered stability issues rather than direct security flaws, attackers can exploit them to trigger denial-of-service attacks. By carefully crafting input data or triggering specific execution paths within the application, attackers may intentionally cause the program to dereference a NULL pointer, resulting in repeated system failures.

Additionally, in some rare cases involving kernel-level or embedded systems, NULL pointer dereference vulnerabilities may be exploited to gain elevated privileges or execute arbitrary code if memory protections are not properly enforced. For this reason, software developers must treat pointer-related vulnerabilities with serious attention and implement strong validation mechanisms within the application.

Business Impact

The presence of NULL pointer dereference vulnerabilities can significantly impact the reliability and stability of software systems. When applications crash due to invalid memory access, users may experience unexpected interruptions while performing critical operations. For organizations that rely on software services for daily business activities, such interruptions can reduce productivity, disrupt workflows, and lead to operational inefficiencies.

One major consequence of this vulnerability is system downtime. If a server application crashes due to a NULL pointer dereference error, the service may become unavailable until it is restarted or repaired. In high-availability environments such as financial platforms, ecommerce websites, or enterprise applications, even short periods of downtime can result in financial losses and damage to the organization’s reputation.

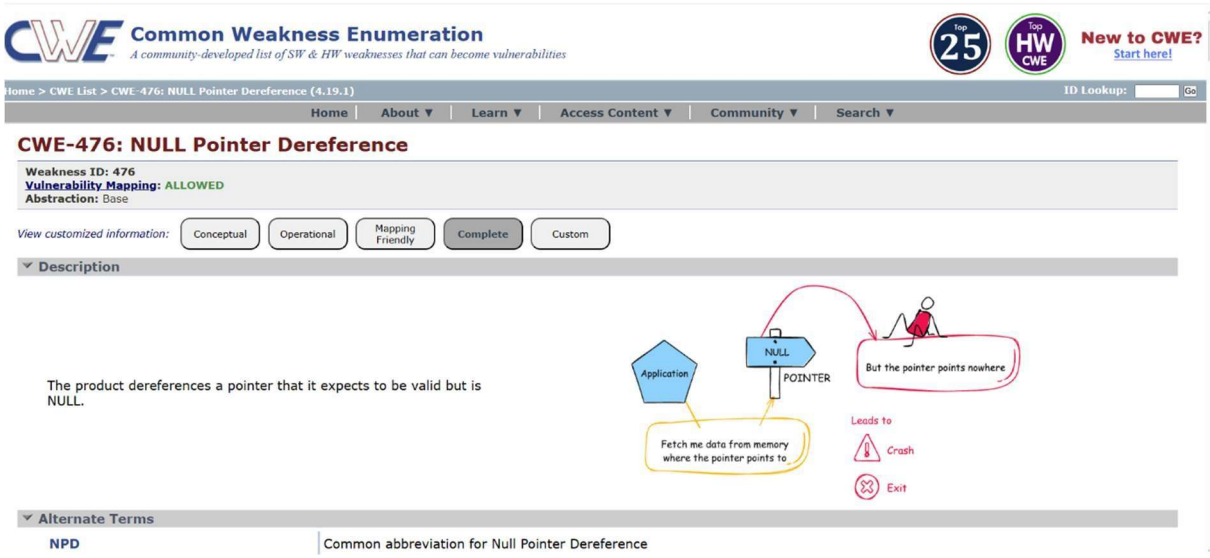
Another potential business risk involves denial-of-service attacks. Attackers who discover a NULL pointer dereference vulnerability may intentionally trigger it repeatedly in order to crash the application. By continuously exploiting the vulnerability, attackers can prevent legitimate

users from accessing the system. This type of attack can severely impact customer experience and may require significant time and resources to investigate and mitigate.

Organizations may also face increased maintenance and development costs when dealing with such vulnerabilities. Debugging memory-related errors can be complex and timeconsuming, especially in large software systems with extensive codebases. Developers may need to analyze system logs, trace memory operations, and redesign portions of the code to prevent the vulnerability from occurring again.

Furthermore, repeated application crashes may negatively affect customer trust and brand reputation. Users expect modern applications to function reliably without unexpected failures. If customers frequently experience system crashes or service interruptions, they may lose confidence in the organization's ability to provide stable and secure services.

For these reasons, organizations must adopt secure development practices and implement thorough testing procedures to identify pointer-related vulnerabilities before deploying software into production environments. Proper input validation, error handling, memory management techniques, and code reviews are essential to preventing NULL pointer dereference issues and ensuring the long-term reliability of software systems.



.

REPORT

6.Vulnerability Name

Persistent Cookies Containing Sensitive Information

CWE/CVE

The vulnerability identified in this assessment corresponds to CWE-539: Use of

Persistent Cookies Containing Sensitive Information, which is a security weakness recognized within the Common Weakness Enumeration (CWE) framework. CWE-539 describes a scenario where sensitive information is stored inside persistent cookies that remain on the client device even after the browser session has ended. Persistent cookies differ from session cookies in that they are not automatically deleted when the browser closes and can remain stored for a long period of time depending on their expiration settings. Because these cookies are stored locally on the user's system, they may be accessed by attackers who gain access to the device, compromise the browser environment, or exploit other vulnerabilities that allow them to read client-side data. If sensitive information such as authentication credentials, session tokens, or personal user data is stored within these cookies without proper protection mechanisms such as encryption or secure attributes, attackers may retrieve this information and use it to impersonate legitimate users or gain unauthorized access to restricted areas of the application. This vulnerability highlights the importance of secure data storage practices and proper session management mechanisms in web application development.

OWASP/SANS Category

This vulnerability is closely related to security risks described in the OWASP Top 10

2021, particularly under A02: Cryptographic Failures and A07: Identification and Authentication Failures. Cryptographic failures occur when sensitive data is not properly protected during storage or transmission, while identification and authentication failures involve weaknesses in session management and authentication mechanisms that may allow attackers to compromise user identities. Storing sensitive information in persistent cookies without proper protection mechanisms can expose critical authentication data and increase the likelihood of unauthorized access.

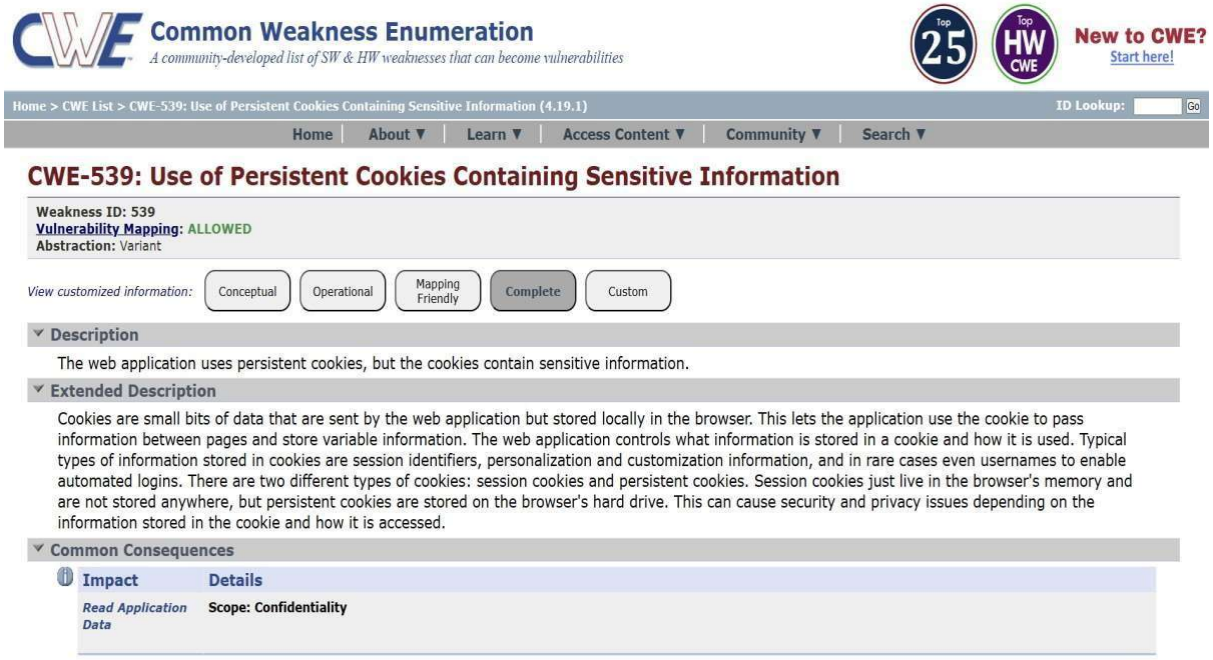
Additionally, this vulnerability aligns with issues highlighted in the SANS Top 25 Most Dangerous Software Errors, which identifies improper handling of sensitive information and insecure storage of data as some of the most common and dangerous programming errors in modern software applications. Organizations that fail to properly secure cookies and other client-side storage mechanisms may unintentionally expose confidential data to attackers, increasing the risk of system compromise and data breaches.

Description

Cookies are small pieces of information that are created by web servers and stored within a user's web browser to maintain stateful interactions between the client and the server. They play an essential role in modern web applications by enabling features such as user authentication, session management, personalization, and tracking of user preferences. There are generally two types of cookies used in web applications: session cookies and persistent cookies. Session cookies are temporary and exist only during the active browser session, meaning they are automatically deleted when the browser is closed. Persistent cookies, on the other hand, are stored on the user's device for a predefined period of time and remain accessible even after the browser is closed. While persistent cookies can improve usability by remembering user preferences and login states, storing sensitive information in them can introduce serious security risks. If sensitive data such as authentication tokens, user identifiers, personal information, or session IDs is stored within persistent cookies, attackers may retrieve this information through various attack techniques including malware infections, cross-site scripting attacks, browser vulnerabilities, or direct access to the user's device. Once the sensitive data is obtained, attackers may use it to impersonate legitimate users, bypass authentication mechanisms, or gain unauthorized access to protected resources within the application. This vulnerability demonstrates the importance of ensuring that sensitive data is never stored in persistent cookies unless it is properly protected and secured using appropriate security mechanisms.

Business Impact

The presence of persistent cookies containing sensitive information can have significant consequences for organizations, users, and the overall security posture of the application. If attackers successfully exploit this vulnerability, they may gain unauthorized access to user accounts and sensitive system resources, leading to potential data breaches and exposure of confidential information. Such incidents can result in financial losses, regulatory penalties, and legal consequences for organizations that fail to adequately protect user data. Additionally, compromised user accounts may be used by attackers to perform fraudulent activities, manipulate application data, or carry out further attacks within the system. Beyond the direct technical impact, security breaches caused by this vulnerability can significantly damage an organization's reputation and erode customer trust. Users expect their personal and sensitive information to be handled securely, and any failure to protect this data may result in loss of business, negative publicity, and long-term reputational harm. Furthermore, organizations may face compliance violations with data protection regulations such as GDPR, HIPAA, or other industry-specific standards if sensitive information is improperly stored and exposed. Therefore, addressing this vulnerability is critical to maintaining strong security practices, protecting user privacy, and ensuring the long-term reliability and credibility of the web application.



REPORT

7. Vulnerability Name

Unexpected Status Code or Return Value

CWE / CVE CWE-394

OWASP / SANS Category Mapped conceptually to OWASP Foundation Top 10 – A03: Injection / Improper Error Handling

Description

The Unexpected Status Code or Return Value vulnerability occurs when an application does not properly validate or handle return values from functions, APIs, system calls, or external services. In software systems, many operations return specific status codes or values that inform the program whether the operation was successful, partially successful, or encountered an error condition. If the application only checks for expected values and ignores other possible valid return values, it may fail to properly handle unusual but legitimate responses.

This situation may cause the application to behave incorrectly or enter an unexpected state. For example, a function may return a value indicating that an operation completed with warnings, partial success, or a special condition. If the application assumes only success or failure outcomes and does not account for other responses, it may incorrectly process data or continue execution in an unsafe manner. In some cases, attackers may exploit this weakness by triggering specific conditions that cause the system to receive unusual return values, potentially bypassing security checks or causing the application to behave unpredictably.

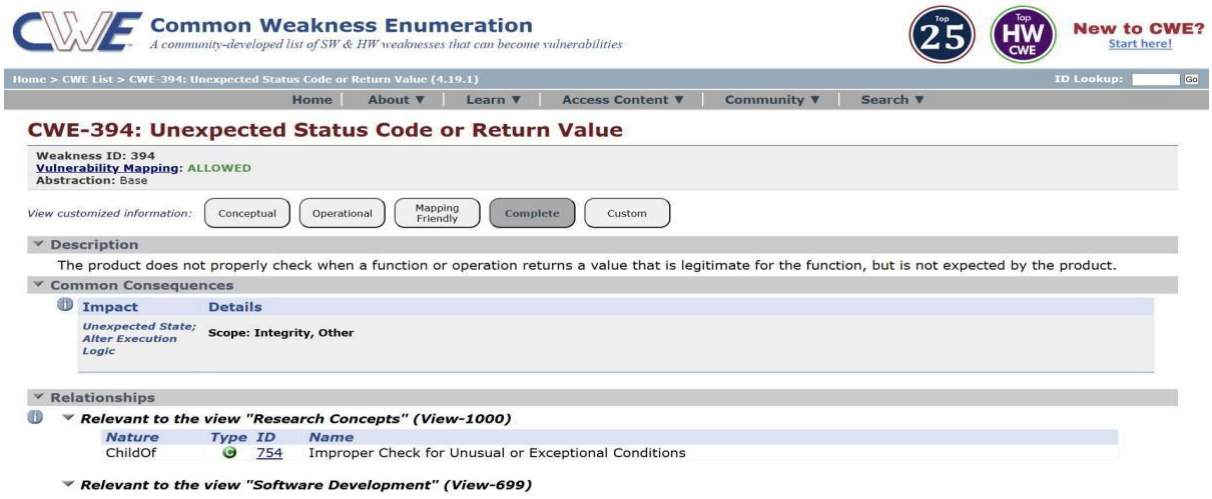
This vulnerability commonly occurs in complex systems that interact with external APIs, databases, network services, or operating system functions. Inadequate error handling, lack of input validation, and incomplete understanding of function behaviors are common causes. Proper defensive programming techniques, comprehensive error checking, and validation of all possible return values are necessary to prevent such vulnerabilities.

Business Impact

The business impact of the Unexpected Status Code or Return Value vulnerability can be significant because it affects the reliability, integrity, and security of software systems. When applications fail to properly handle unexpected responses from functions or external systems, they may behave unpredictably or perform incorrect operations. This can lead to system crashes, corrupted data, or incorrect transaction processing, which directly affects the quality and reliability of the organization's services.

In security-sensitive environments such as financial systems, healthcare platforms, or enterprise applications, improper handling of return values could allow attackers to manipulate system behavior or bypass important validation mechanisms. This may result in unauthorized access to sensitive information, incorrect authorization decisions, or disruption of critical services. Additionally, unpredictable system behavior may lead to operational downtime, affecting business productivity and customer trust.

Organizations that rely on complex software infrastructures may also face increased costs associated with troubleshooting, incident response, and system recovery when unexpected errors occur. If such issues lead to service outages or data integrity problems, the organization may experience reputational damage and potential financial losses. Therefore, ensuring robust error handling and comprehensive validation of all return values is essential for maintaining system stability, protecting sensitive data, and supporting reliable business operations.



REPORT

8. Vulnerability Name

Insufficient Logging

CWE / CVE CWE-778

OWASP / SANS Category Mapped to OWASP Foundation Top 10 – A09: Security Logging and Monitoring Failures

Description

The Insufficient Logging vulnerability occurs when an application or system fails to properly record important security-related events or logs them without sufficient details for analysis. Logging is a critical security mechanism that helps track system activities and detect malicious behavior. However, when logs are incomplete, missing, or not generated for key events, it becomes extremely difficult to monitor the system effectively or respond to security incidents.

For example, if an application does not log failed login attempts, privilege changes, or suspicious user activities, administrators will not be able to detect brute-force attacks, unauthorized access attempts, or abnormal system behavior. Even when logs exist, they may lack important contextual information such as timestamps, user identifiers, IP addresses, or event descriptions, making it difficult to reconstruct the sequence of events during an investigation.

In many cloud-based and enterprise environments, logging features are sometimes disabled by default or require additional configuration. Organizations may also limit logging due to storage costs or performance concerns. However, these limitations can create gaps in security monitoring, leaving systems vulnerable to undetected attacks. Without adequate logs, security teams cannot perform effective forensic analysis or identify the source of security incidents.

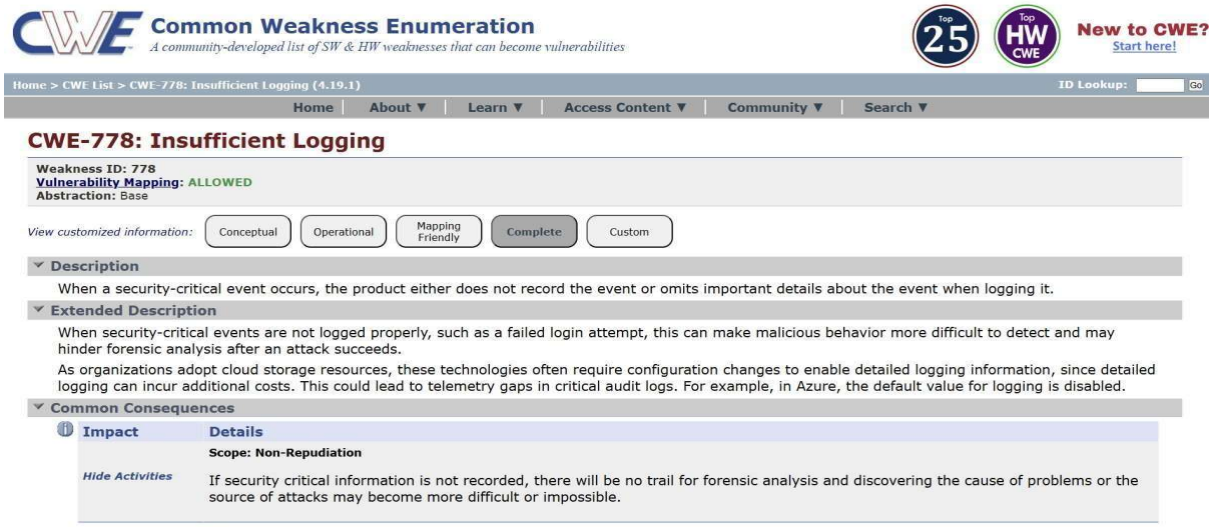
Proper logging should capture detailed information about authentication events, system errors, configuration changes, user activities, and security alerts. Logs should also be securely

stored, monitored regularly, and integrated with security monitoring systems such as SIEM tools to ensure early detection of threats.

Business Impact

The business impact of insufficient logging can be severe for organizations because it directly affects their ability to detect, investigate, and respond to cyber threats. When security events are not properly logged, attackers can perform malicious activities within the system without leaving any trace of their actions. This means organizations may not even realize that a security breach has occurred until significant damage has already been done.

Without proper logging, incident response teams cannot determine how an attack happened, which systems were affected, or what data may have been compromised. This lack of visibility makes forensic investigations extremely difficult and delays the organization's ability to contain and recover from security incidents. As a result, attackers may remain inside the network for extended periods of time, increasing the risk of data theft, system manipulation, or service disruption.



REPORT

9.Vulnerability Name

Inclusion of Web Functionality from an Untrusted Source

CWE / CVE CWE-830

OWASP / SANS Category Mapped to OWASP Foundation Top 10 – A08: Software and Data Integrity Failures

Description

The vulnerability occurs when a web application includes or executes external functionality, such as JavaScript files, widgets, plugins, or other resources, from a domain that is not fully trusted or verified. When these external components are loaded, they execute within the context of the hosting application's domain. This means the external code may gain the same privileges as the application itself.

In many modern web environments, developers include third-party functionality using HTML <script> tags or external APIs to add features like analytics tracking, user interface enhancements, advertisements, or social media integrations. However, if the external domain is compromised, manipulated during transmission, or intentionally malicious, it can inject harmful code into the application.

Because the browser executes the external code with the same permissions as local scripts, the malicious functionality can access sensitive information such as session tokens, cookies, or user input fields. It may also modify page content, redirect users to phishing sites, or perform unauthorized actions on behalf of the user. This vulnerability is especially common in mashup applications where multiple external sources are integrated without strict security controls.

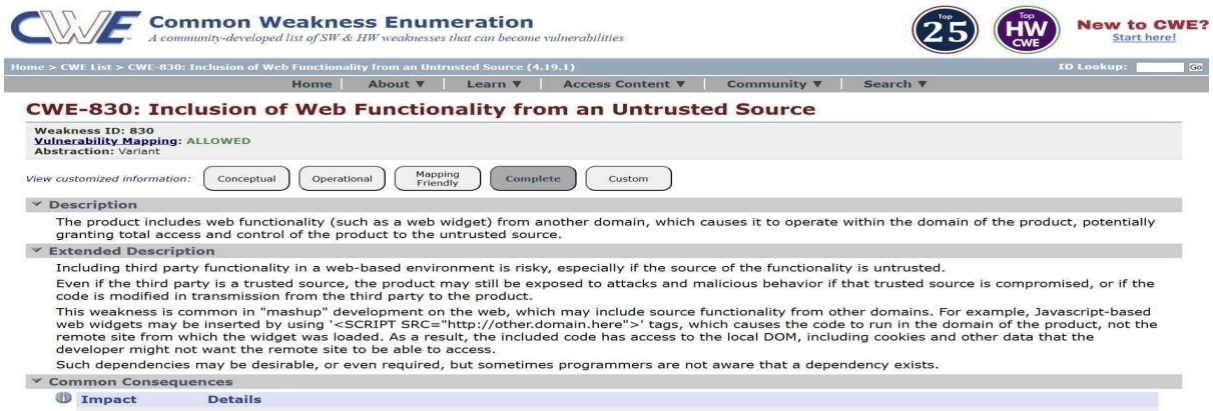
Developers often overlook this risk because the external functionality appears legitimate at the time of integration. However, if the source later becomes compromised or the script is modified, all applications relying on it become vulnerable. Proper security mechanisms such

as Content Security Policy (CSP), Subresource Integrity (SRI), and strict domain whitelisting are necessary to reduce the risks associated with this vulnerability.

The inclusion of web functionality from an untrusted source can have severe consequences for organizations, users, and business operations. If an attacker compromises the external resource or injects malicious code into a third-party script, the malicious code will automatically execute in every user's browser when they access the affected application. This can lead to large-scale data breaches, unauthorized access to sensitive user information, and manipulation of application functionality. Attackers may steal authentication cookies, enabling them to hijack user sessions and gain access to accounts without needing passwords.

Additionally, malicious scripts could modify web content to redirect users to phishing websites, distribute malware, or display fraudulent information. This not only compromises the security of users but also damages the organization's reputation and trustworthiness. Customers who experience security incidents may lose confidence in the organization's ability to protect their data, potentially resulting in financial losses, legal consequences, and regulatory penalties. In large-scale systems such as ecommerce platforms, banking services, or enterprise applications, this vulnerability could disrupt business operations and expose confidential corporate data.

Furthermore, organizations may face compliance violations if sensitive information such as personal data, financial records, or confidential business information is exposed through compromised external scripts. The cost of incident response, system recovery, legal liabilities, and reputation management can be extremely high. Therefore, organizations must implement strict security policies when integrating third-party



REPORT

10. Vulnerability Name

Use of Default Credentials

CWE/CVE

The vulnerability identified in this assessment corresponds to CWE-1392: Use of Default Credentials. This weakness occurs when a product or system relies on predefined authentication credentials such as default usernames, passwords, or cryptographic keys for critical functionality. These credentials are typically embedded within the system to simplify the setup and deployment process. However, if administrators fail to modify these credentials after installation, attackers can easily exploit them to gain access to the system. Since many default credentials are publicly documented in product manuals or online databases, attackers often use automated tools to scan for systems that still use default login information. Once an attacker successfully logs in using these credentials, they may gain access to sensitive resources, administrative controls, or internal system configurations. This vulnerability highlights the importance of enforcing secure authentication practices and ensuring that all default credentials are replaced with strong, unique passwords before a system is placed into production.

OWASP/SANS Category

This vulnerability is closely related to security issues described in the OWASP Top 10 2021, particularly A07: Identification and Authentication Failures. This category addresses weaknesses in authentication mechanisms that may allow attackers to bypass login controls or gain unauthorized access to systems and applications. The use of default credentials directly contributes to authentication failures because it allows attackers to log in using widely known or easily guessable passwords. Additionally, this issue aligns with the security risks highlighted in the SANS Top 25 Most Dangerous Software Errors, which identifies improper authentication and weak credential management as major sources of security vulnerabilities in modern software systems. Organizations that fail to enforce strong password

policies or neglect to change default credentials during system deployment significantly increase the risk of unauthorized access and system compromise.

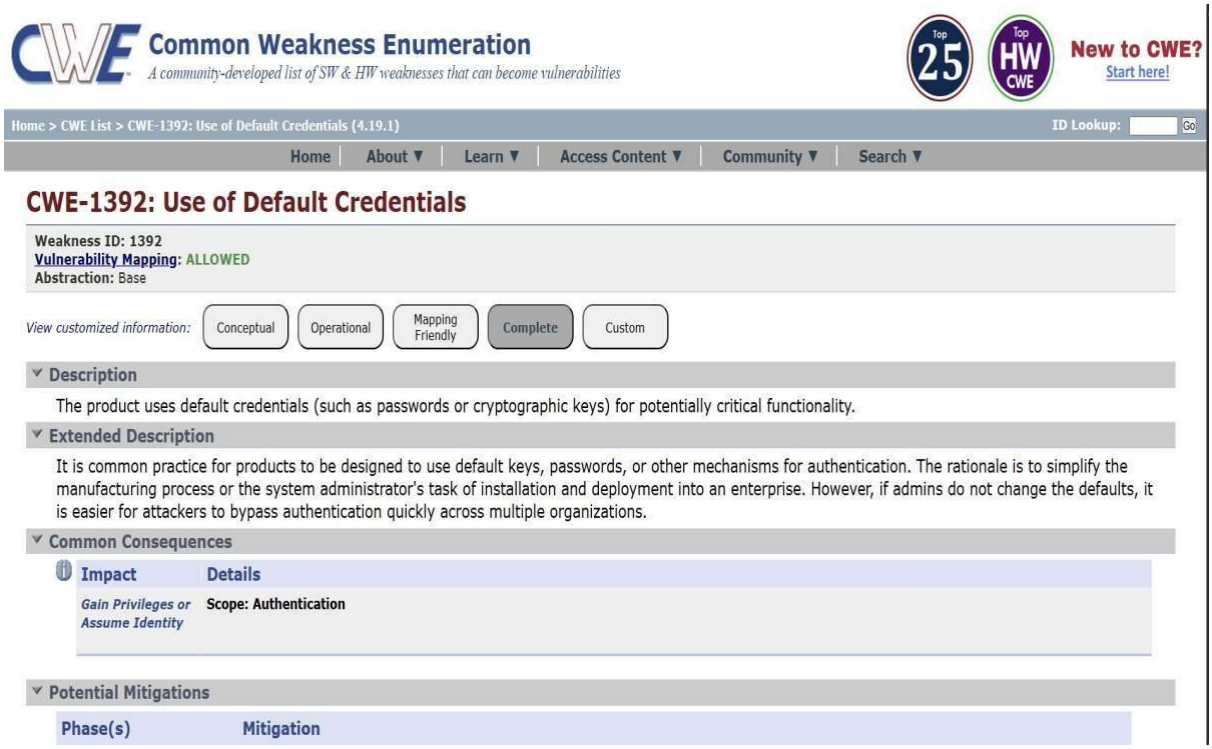
Description

Default credentials are commonly included in software products, network devices, and web applications to simplify the installation and configuration process for administrators and system integrators. These credentials typically consist of predefined usernames and passwords that allow administrators to access the system during the initial setup phase. While this practice may improve convenience during deployment, it also introduces significant security risks if the default credentials are not changed afterward. Attackers are well aware of common default credentials used by various systems and often maintain databases of these credentials for exploitation purposes. In many cases, automated scanning tools are used to identify systems that still rely on default login information. Once an attacker successfully logs in using default credentials, they may gain access to critical system functionality, sensitive data, or administrative controls. This access can allow them to modify system configurations, install malicious software, or extract confidential information from the system. The vulnerability becomes particularly dangerous in enterprise environments where systems are connected to internal networks and contain sensitive organizational data. Because default credentials are easily obtainable and widely known, their continued use in production systems creates a serious security weakness that must be addressed through proper configuration management and secure authentication practices.

Business Impact

The use of default credentials within a system can lead to severe security and operational consequences for an organization. If attackers exploit this vulnerability, they may gain unauthorized access to sensitive systems, applications, or databases without needing sophisticated attack techniques. This unauthorized access may allow attackers to steal confidential information, manipulate system data, disrupt business operations, or deploy malware within the organization's network. In many cases, attackers who gain administrative access through default credentials can maintain persistent control over the system and launch additional attacks against other connected systems. Such security breaches can lead to financial losses, operational downtime, and potential legal liabilities for the affected

organization. Additionally, organizations may face regulatory penalties if sensitive customer or business data is compromised due to poor security practices. Beyond the financial and legal consequences, security incidents resulting from default credential exploitation can significantly damage an organization's reputation and reduce customer trust. Clients and stakeholders expect organizations to maintain strong security controls to protect sensitive information, and failures in this area can result in long-term reputational harm. Therefore, addressing this vulnerability is essential to maintaining system integrity, protecting organizational assets, and ensuring compliance with industry security standards and best practices.



STAGE 2

Overview :-

This report presents the results of a web application vulnerability assessment. The assessment identified several security weaknesses that could allow attackers to compromise the confidentiality, integrity, and availability of the system. The vulnerabilities mainly affect web application input validation, authentication mechanisms, configuration settings, and data protection practices. Addressing these issues will reduce the risk of data breaches, unauthorized access, and system compromise.

Target website

Not specified in the provided document.

Target ip address:-

Not specified in the provided document.

List of vulnerability:

S.No	Vulnerability Name	Severity	Port	Description
1	SQL Injection	High	80 / 443	Unsanitized user input allows attackers to manipulate SQL queries and access or modify database data.
2	Cross-Site Scripting (XSS)	High	80 / 443	Attackers inject malicious scripts into web pages that execute in users' browsers.

3	Insecure Direct Object References (IDOR)	Medium	80 / 443	Users can access unauthorized data by modifying object identifiers in requests.
4	Directory Traversal	Medium	80	Attackers manipulate file paths to access restricted files or directories.
5	Broken Authentication	High	443	Weak authentication and session management allow attackers to impersonate users.
6	Security Misconfiguration	Low	443	Default settings or improper configurations expose the system to attacks.
7	Cross-Site Request Forgery (CSRF)	Low	8080 / 443	Attackers trick authenticated users into performing unwanted actions.
8	Unvalidated Redirects and Forwards	Medium	80 / 443	Users can be redirected to malicious websites through manipulated URLs.
9	Sensitive Data Exposure	High	443	Sensitive information such as passwords or financial data may be exposed due to weak protection mechanisms.

Report

The vulnerability scan identified multiple high, medium, and low severity vulnerabilities in the web application. High-risk issues such as SQL Injection, Cross-Site Scripting, Broken Authentication, and Sensitive Data Exposure may allow attackers to access or manipulate sensitive data, hijack sessions, or take control of user accounts. Medium-severity issues like IDOR, Directory Traversal, and Unvalidated Redirects could allow unauthorized data access or phishing attacks. Low-severity issues such as Security Misconfiguration and CSRF increase the attack surface and may assist attackers in exploiting other vulnerabilities. Immediate remediation steps such as implementing secure coding practices, proper input validation, strong authentication controls, and secure configuration management are recommended.

1. VULNERABILITY NAME:

SEVERITY: High

PLUGIN: Input Validation Failure

PORT: 80/443

DESCRIPTION: Unsanitized user input in query parameters allows attackers to manipulate SQL queries, potentially exposing or altering sensitive data.

SOLUTION:Use parameterized queries and stored procedures.

Employ ORM frameworks with built-in protections.

Regularly test with automated scanners.

BUSINESS IMPACT: Data theft, unauthorized account access, and reputational damage.

2. VULNERABILITY NAME: Cross-Site Scripting (XSS)

SEVERITY: High

PLUGIN: Reflected/Stored XSS Detection

PORT: 80/443

DESCRIPTION: Malicious scripts can be injected into web pages viewed by other users.

SOLUTION:Encode output properly.

Use Content Security Policy (CSP).

Sanitize all user inputs.

BUSINESS IMPACT: Session hijacking, defacement, and loss of customer trust.

3. VULNERABILITY NAME: Insecure Direct Object References (IDOR)

SEVERITY: Medium

PLUGIN: Authorization Bypass Detection

PORT: 80/443

DESCRIPTION: Users can access objects by manipulating identifiers in requests without proper authorization checks.

SOLUTION:Enforce access control checks at every layer.

Use indirect references instead of exposing raw IDs.

BUSINESS IMPACT: Unauthorized data exposure and compliance violations.

4. VULNERABILITY NAME: Directory Traversal

SEVERITY: Medium

PLUGIN: Path Manipulation Detection

PORT: 80

DESCRIPTION: Attackers can manipulate file paths to access restricted directories or files.

SOLUTION:Normalize and validate input paths.

Restrict file system access.

BUSINESS IMPACT: Leakage of sensitive configuration files.

5. VULNERABILITY NAME: Broken Authentication

SEVERITY: High

PLUGIN: Weak Session Management

PORT: 443

DESCRIPTION: Poorly implemented authentication mechanisms allow attackers to impersonate users.

SOLUTION:Enforce MFA.

Use secure session tokens with expiration.

BUSINESS IMPACT: Account takeover and fraud.

6. VULNERABILITY NAME: Security Misconfiguration

SEVERITY: Low

PLUGIN: Default Credentials/Weak Configurations

PORT: 443

DESCRIPTION: Default settings or unnecessary services expose systems to attack.

SOLUTION:Harden configurations.

Remove unused services.

BUSINESS IMPACT: Increased attack surface.

7. VULNERABILITY NAME: Cross-Site Request Forgery (CSRF)

SEVERITY: Low

PLUGIN: HTTP Form-Based Authentication Detected

PORT: 8080/443

DESCRIPTION: Attackers trick authenticated users into executing unwanted actions.

SOLUTION:Implement anti-CSRF tokens.

Use SameSite cookie attributes.

BUSINESS IMPACT: Unauthorized transactions and compromised accounts.

9. VULNERABILITY NAME: Unvalidated Redirects and Forwards

SEVERITY: Medium

PLUGIN: URL Redirection Detection

PORT: 80/443

DESCRIPTION: Attackers can redirect users to malicious sites via unvalidated parameters.

SOLUTION:Validate and whitelist redirect targets.

Avoid using user-supplied input for redirects.

BUSINESS IMPACT: Phishing and malware distribution.

10. VULNERABILITY NAME: Sensitive Data Exposure

SEVERITY: High

PLUGIN: Information Disclosure Detection

PORT: 443

DESCRIPTION: Improper handling of sensitive data (e.g., passwords, credit card numbers) exposes it to attackers.

SOLUTION:Encrypt data at rest and in transit.

Mask sensitive fields in logs.

BUSINESS IMPACT: Identity theft, financial loss, and legal penalties.

STAGE 3

Title:

Analysis of Security Monitoring Capabilities Using SOC, SIEM and QRadar

SOC

A Security Operations Center (SOC) is a centralized team responsible for continuously monitoring, detecting, analyzing, and responding to cybersecurity incidents within an organization. The SOC team uses various security tools, threat intelligence, and monitoring platforms to identify suspicious activities in the network. Their main objective is to protect organizational systems, data, and infrastructure from cyber threats such as malware, phishing attacks, data breaches, and unauthorized access. SOC teams work 24/7 to ensure that any potential security incident is quickly detected and mitigated before it causes serious damage to the organization.

SOC Cycle

The SOC cycle represents the continuous process followed by the Security Operations Center to manage cybersecurity threats. It typically includes stages such as monitoring, detection, investigation, response, and recovery. First, the SOC continuously monitors network activities and system logs using security tools. When suspicious behavior is detected, analysts investigate the incident to determine whether it is a real threat. If confirmed, appropriate response actions such as blocking malicious IP addresses or isolating compromised systems are taken. Finally, the SOC performs recovery actions and improves security controls to prevent similar attacks in the future.

SIEM

Security Information and Event Management (SIEM) is a security solution that collects, analyzes, and correlates security logs and events from multiple sources within an organization's IT environment. SIEM tools help security teams identify unusual patterns or malicious activities by analyzing large amounts of log data in real time. These tools provide centralized monitoring of networks, servers, applications, and devices. SIEM solutions are widely used in

SOC environments because they allow security teams to detect threats quickly and respond effectively to potential cyber attacks.

SIEM Cycle

The SIEM cycle involves several important stages that ensure effective security monitoring and threat detection. The first stage is data collection, where logs and events are gathered from various sources such as firewalls, servers, network devices, and applications. The second stage is normalization and correlation, where the system analyzes and compares events to identify suspicious patterns. The third stage is alert generation, where SIEM tools notify security analysts about potential threats. Finally, security teams investigate alerts and respond to incidents by taking necessary actions to mitigate the risk.

MISP

MISP (Malware Information Sharing Platform) is an open-source threat intelligence platform designed for sharing information about cybersecurity threats. It allows organizations to exchange information related to malware indicators, attack techniques, malicious IP addresses, and threat actors. By sharing threat intelligence, organizations can better understand emerging threats and improve their security defenses. MISP helps security teams collaborate and detect cyber attacks faster by providing a centralized platform for threat intelligence analysis and sharing.

.

Threat Intelligence

Threat intelligence refers to the collection and analysis of information about potential or existing cyber threats. It helps organizations understand attacker behavior, attack techniques, and emerging vulnerabilities. Threat intelligence data may include malicious IP addresses, malware signatures, phishing domains, and hacker group activities. Security teams use this information to improve their defensive strategies and prevent attacks before they occur. Integrating threat intelligence with SIEM systems allows organizations to identify threats more effectively and respond quickly to potential security incidents.

Your College Network Information

The network infrastructure of A.V.P College of Arts and Science for Women consists of multiple interconnected systems such as administrative office computers, computer laboratories used by students, faculty systems, servers, Wi-Fi networks, and internet gateways. These networks support several academic and administrative activities including online learning platforms, digital library access, email communication, examination management systems, and institutional databases. Since a large number of students, faculty members, and staff access the network daily, maintaining strong cybersecurity measures is essential. Security controls such as firewalls, antivirus systems, access control mechanisms, and network monitoring tools help protect sensitive academic records, student information, and administrative data from cyber threats such as malware attacks, unauthorized access, and data breaches.

How You Think You Deploy SOC in Your College

To enhance cybersecurity in A.V.P College of Arts and Science for Women, a Security Operations Center (SOC) could be implemented to monitor and manage the college network infrastructure. The SOC would collect logs and security events from various devices such as servers, routers, firewalls, and endpoint computers using a SIEM platform. Security analysts within the SOC would continuously monitor these logs to identify suspicious activities like repeated login failures, unusual network traffic, or unauthorized access attempts. Additionally, the SOC could integrate threat intelligence sources, automated alert systems, and incident response procedures to quickly detect and respond to potential cyber threats. Implementing a SOC would significantly improve the college's ability to safeguard student data, academic systems, and institutional resources from cyber attacks.

Incident Response

Incident response is the process of identifying, managing, and resolving cybersecurity incidents within an organization. When a security breach or suspicious activity is detected, incident response teams follow structured procedures to contain the threat and minimize damage. The process usually includes identification, containment, eradication, recovery, and post-incident analysis. Effective incident response helps organizations restore normal operations quickly and prevent similar attacks in the future by improving their security defenses.

QRadar & Understanding About Tool

IBM QRadar is a widely used SIEM platform that helps organizations monitor security events and detect cyber threats in real time. QRadar collects logs and network flow data from multiple devices across the organization and analyzes them to identify suspicious activities. The tool uses advanced analytics and correlation rules to generate alerts for potential security incidents. Security analysts use the QRadar dashboard to investigate threats, track attack patterns, and respond to incidents efficiently. By providing centralized security visibility, QRadar plays an important role in modern SOC environments.

Conclusion

Stage 1 – What I understood from Web Application Testing

From Stage 1, I understood the importance of identifying vulnerabilities in software applications. Web application testing helps security professionals detect weaknesses such as SQL Injection, Cross-Site Scripting, and authentication issues. By understanding CWE standards and common vulnerabilities, developers can implement secure coding practices and reduce security risks in applications.

Stage 2 – What I understood from the Nessus Report

In Stage 2, the vulnerability assessment report showed how automated security scanning tools can identify weaknesses in web applications. The report highlighted vulnerabilities such as SQL Injection, Broken Authentication, and Sensitive Data Exposure, which can lead to serious security breaches if not addressed. It also demonstrated the importance of vulnerability management and proper security configurations in protecting systems from cyber attacks.

Stage 3 – What I understood from SOC / SIEM / QRadar Dashboard

From Stage 3, I learned how SOC teams use SIEM tools like QRadar to monitor network activity and detect potential threats. The dashboard provides real-time visibility into security events and helps analysts investigate suspicious activities. This stage highlighted the importance of centralized monitoring, threat intelligence integration, and incident response in modern cybersecurity operations.

Future Scope

Stage 1 – Future Scope of Web Application Testing

The future of web application testing will involve more advanced automated security tools, AIbased vulnerability detection, and secure development practices. Continuous security testing during the development lifecycle will help organizations identify vulnerabilities earlier and reduce the risk of cyber attacks.

Stage 2 – Future Scope of Testing Process

The vulnerability testing process will continue to improve with advanced scanning technologies, automated risk assessment tools, and integration with DevSecOps environments. Organizations will adopt continuous vulnerability monitoring to ensure that systems remain secure against evolving cyber threats.

Stage 3 – Future Scope of SOC / SIEM

The future of SOC and SIEM technologies will include artificial intelligence, machine learning, and automated threat detection systems. These technologies will help security teams detect attacks faster and respond to incidents more efficiently. Cloud-based SIEM platforms and automated security orchestration tools will further improve cybersecurity operations.